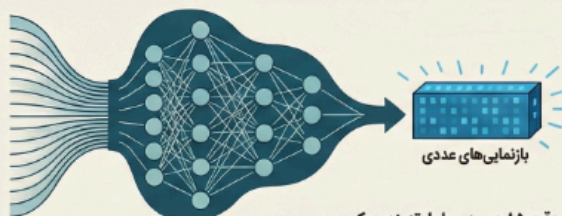


بخش ۲: استفاده از مدل‌های زبانی از پیش آموزش داده شده

راهنمای استراتژی‌های طبقه‌بندی متن با مدل‌های زبانی (PLMs)

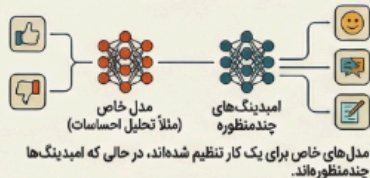
معرفی و مقایسه روش‌های مختلف استفاده از مدل‌های زبانی پیش‌آموزش‌دیده برای طبقه‌بندی متن.

بخش ۱: مدل‌های بازنمایی (Representation Models)



دقت ۸۵ درصدی با طبقه‌بند سبک

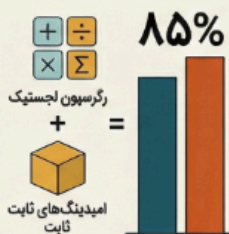
مدل خاص وظیفه در مقابل مدل امیدینگ



مدل‌های خاص برای یک کار تنظیم شده‌اند، در حالی که امیدینگ‌ها چندمنظوره‌اند.

کارایی بالا در معماری فقط رمزگذار

مدل‌هایی مثل BERT و RoBERTa به دلیل حجم کمتر برای وظایف تخصصی بسیار بهینه هستند.



ترکیب امیدینگ‌های ثابت با رگرسیون لجستیک می‌تواند عملکردی بهتر از مدل‌های تنظیم‌شده داشته باشد.

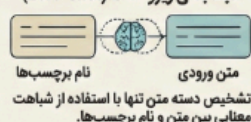
بخش ۲: مدل‌های مولد و روش‌های نوین (Generative & Zero-Shot)

طبقه‌بندی با مهندسی پرامپت



طبقه‌بندی با مهندسی پرامپت در مدل‌های مولد، هدف از طریق دستورالعمل (Prompt) به مدل منتقل می‌شود.

طبقه‌بندی زیر-شات (Zero-Shot)



مدل‌های متن‌باز (T5) در برابر اختصاصی (GPT)



مدل‌های مولد: تولید متن و طبقه‌بندی بدون داده



مقایسه عملکرد (F1-Score) روش‌های مختلف روی مجموعه‌داده نقد فیلم (Rotten Tomatoes)



فصل ۴: طبقه‌بندی متن (Text Classification)

یکی از کارهای رایج در پردازش زبان طبیعی (Natural Language Processing - NLP)، طبقه‌بندی متن (Text Classification) است. در این روش، هدف آن است که مدلی آموزش داده شود تا بتواند برای یک متن ورودی، برچسب یا دسته‌ای مناسب (Label / Class) انتخاب کند (تصویر زیر)

Movie review	Class
What a horrible movie...	0 negative
Despite some flaws, great experience.	1 positive
Best movie ever!	1 positive

Never want to see this movie again.	?
-------------------------------------	---

طبقه‌بندی متن در کاربردهای گوناگونی استفاده می‌شود؛ مثل تحلیل احساسات (**Sentiment Analysis**)، تشخیص هدف یا نیت کاربر (**Intent Detection**)، استخراج نام‌ها و موجودیت‌ها (**Entity Extraction**)، و حتی شناسایی زبان متن (**Language Detection**).

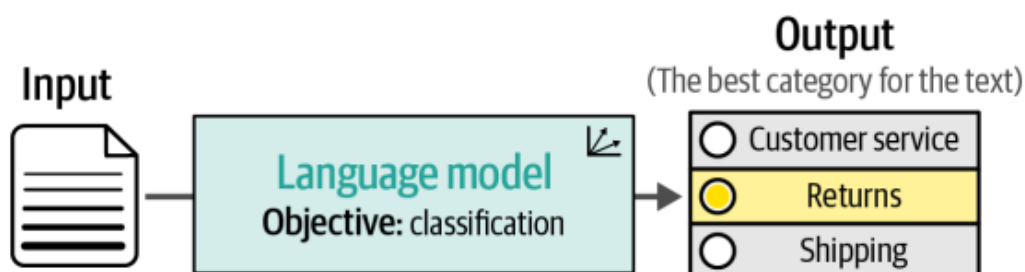
I hear **the Netherlands** is wonderful in the **summer**

Figure 1-17. An example of named entity recognition that detects the entities “place” and “time”.

On **February 11** **DATE** , **2020** **DATE** , a **73-year-old** **AGE** woman came to the **hospital** **LOCATION** and was diagnosed with **COVID-19** **DISEASE** and has a **cough** **SYMPTOM** .

Figure 1-18. The output of SpaCy using OpenAI’s GPT-3.5 model. Without any training, it correctly identifies our custom entities.

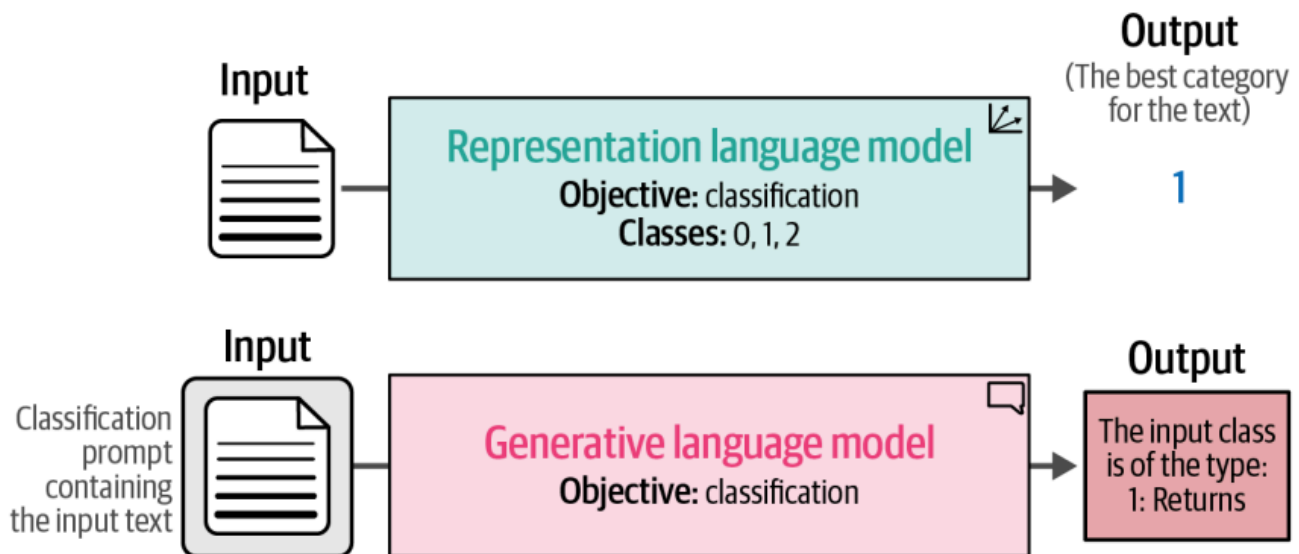
مدل‌های زبانی (**Language Models**) . چه مدل‌های نمایشی (**Representative Models**) و چه مدل‌های تولیدی (**Generative Models**) . نقش بسیار مهم و تعیین‌کننده‌ای در بهبود کیفیت طبقه‌بندی ایفا کرده‌اند.



در این فصل، چندین روش برای استفاده از مدل‌های زبانی (Language Models) در طبقه‌بندی متن (Text Classification) را بررسی خواهیم کرد. هدف این فصل آن است که مقدمه‌ای روشن و قابل فهم برای به کارگیری مدل‌های زبانی از پیش آموزش دیده (Pretrained Language Models) ارائه دهد. از آنجا که طبقه‌بندی متن حوزه‌ای گسترده است، چند رویکرد مهم را بررسی می‌کنیم تا دید بهتری نسبت به توانایی‌های مدل‌های زبانی به دست آوریم:

- «طبقه‌بندی متن با مدل‌های بازنمایی متن» (Text Classification with Representation Models): در این بخش، انعطاف‌پذیری مدل‌های غیرمولد (Nongenerative Models) برای طبقه‌بندی نشان داده می‌شود. در این بخش هم مدل‌های ویژه‌ی یک وظیفه خاص (Task-Specific Models) و هم مدل‌های تعبیه‌ای (Embedding Models) بررسی خواهند شد.
- «طبقه‌بندی متن با مدل‌های مولد (تولیدی)» (Text Classification with Generative Models): در این بخش، مدل‌های زبانی مولد (Generative Language Models) معرفی می‌شوند، زیرا بسیاری از این مدل‌ها را می‌توان برای طبقه‌بندی به کار برد. در این قسمت، هم یک مدل منبع باز (Open Source) و هم یک مدل اختصاصی منبع بسته (Closed Source) بررسی خواهند شد.

تمرکز اصلی این فصل بر مدل‌های از پیش آموزش دیده است یعنی مدل‌هایی که با حجم بسیار بزرگی از داده‌ها آموزش دیده‌اند و می‌توان از توانایی آن‌ها در طبقه‌بندی متن بهره گرفت. همان‌طور که در شکل زیر نشان داده شده، هر دو گروه مدل‌های بازنمایی (Representation Models) و مدل‌های تولیدی (Generative Models) را بررسی کرده و تفاوت‌هایشان را مرور خواهیم کرد.



این فصل به عنوان مقدمه‌ای بر انواع مختلف مدل‌های زبانی، اعم از مولد (Generative) و غیرمولد (Nongenerative)، است. همچنین، در این فصل پکیج‌های رایجی را که برای بارگیری و استفاده از این مدل‌ها به کار می‌روند، بررسی خواهیم کرد.

اگرچه تمرکز این نوشتار بر روی مدل‌های زبانی بزرگ (LLMs) است، اما توصیه می‌شود این مثال‌ها را با روش‌های کلاسیک، اما قوی، مقایسه کنید؛ به عنوان مثال، می‌توان متن را با TF-IDF نمایش داد و بر روی آن یک طبقه‌بند رگرسیون لجستیک (Logistic Regression Classifier) آموزش داد.

تحلیل احساسات در نقدهای سینمایی

برای بررسی روش‌های مختلف در **طبقه‌بندی متن (Text Classification)**، از داده‌هایی استفاده می‌کنیم که در پلتفرم **Hugging Face Hub** قرار دارند؛ این پلتفرم نه تنها میزبان مدل‌ها، بلکه مجموعه داده‌ها (**Datasets**) هم هست.

در اینجا از مجموعه داده‌ی شناخته‌شده‌ی **"rotten_tomatoes"** استفاده خواهیم کرد تا مدل‌های خود را آموزش داده و ارزیابی کنیم. این مجموعه شامل **۵,۳۳۱ نقد مثبت** و **۵,۳۳۱ نقد منفی** از سایت Rotten Tomatoes است.

برای بارگذاری این داده‌ها، از بسته‌ی **datasets** استفاده می‌کنیم؛ بسته‌ای که در طول این نوشتار بارها به کار گرفته خواهد شد.

```
# Rotten Tomatoes بارگذاری مجموعه داده‌ی
from datasets import load_dataset
# Load our data
data = load_dataset("rotten_tomatoes")
data
```

پس از اجرای این کد، داده‌ها به صورت زیر نمایش داده خواهند شد:

```
# ساختار مجموعه داده
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 8530
  })
  validation: Dataset({
    features: ['text', 'label'],
    num_rows: 1066
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 1066
  })
})
```

این داده‌ها به سه بخش **train** (آموزشی)، **test** (آزمون) و **validation** (اعتبارسنجی) تقسیم شده‌اند. در این فصل، هنگام آموزش مدل از بخش **train** استفاده خواهیم کرد و نتایج را با بخش **test** ارزیابی خواهیم کرد.

لازم به یادآوری است که در این مجموعه داده، بخشی به نام **اعتبارسنجی (Validation)** نیز وجود دارد. از این بخش می‌توان برای ارزیابی دقیق‌تر **توانایی تعمیم مدل (Generalization)** استفاده کرد. اهمیت اعتبارسنجی زمانی بیشتر می‌شود که برای **تنظیم ابرپارامترها (Hyperparameter Tuning)** از داده‌های **آموزش (Train)** و **آزمون (Test)** بهره گرفته باشید.

حال، نگاهی به نمونه آخر (-1) از مجموعه داده‌ی آموزشی (**Train**) (اندیس صفر) می‌اندازیم:

```
# نمایش نمونه‌هایی از داده‌ی آموزشی
data["train"][0, -1]
```

خروجی این دستور به صورت زیر خواهد بود:

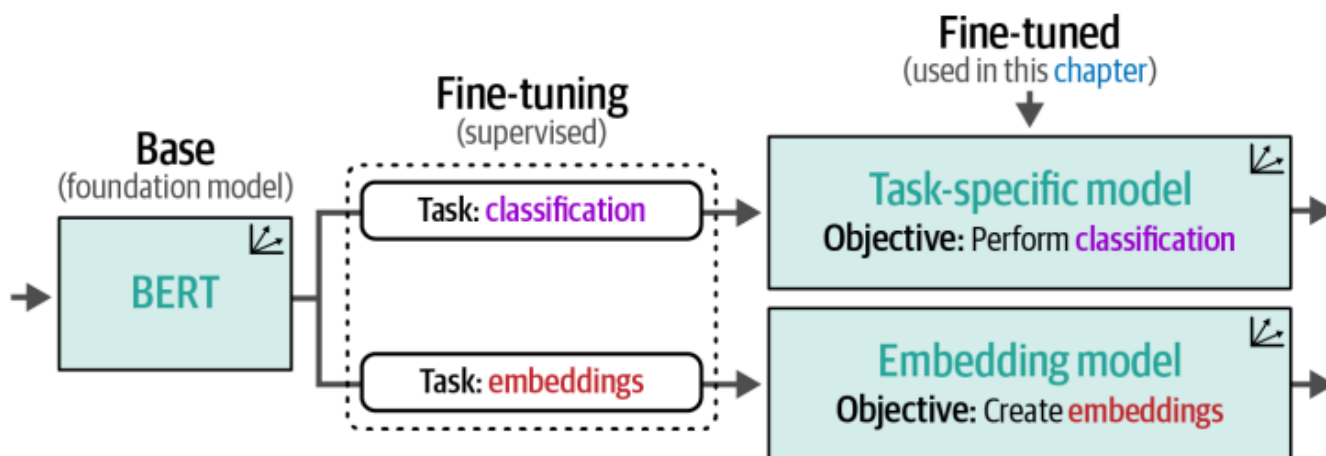
```
# نمونه‌ای از داده‌ها
{'text': ['the rock is destined to be the 21st century\'s new " conan " and
... no content .'],
 'label': [1, 0]}
```

این نقدهای کوتاه با دو برچسب مشخص می‌شوند: **مثبت (۱)** یا **منفی (۰)**. بنابراین در اینجا تمرکز ما بر روی طبقه‌بندی دودویی احساسات (Binary Sentiment Classification) خواهد بود.

طبقه‌بندی متن با مدل‌های بازنمایی

طبقه‌بندی با استفاده از مدل‌های بازنمایی از پیش آموزش‌دیده (Pretrained Representation Models) معمولاً به دو شیوه انجام می‌شود: استفاده از یک مدل خاص وظیفه (Task-Specific Model) یا یک مدل امبدینگ (Embedding Model).

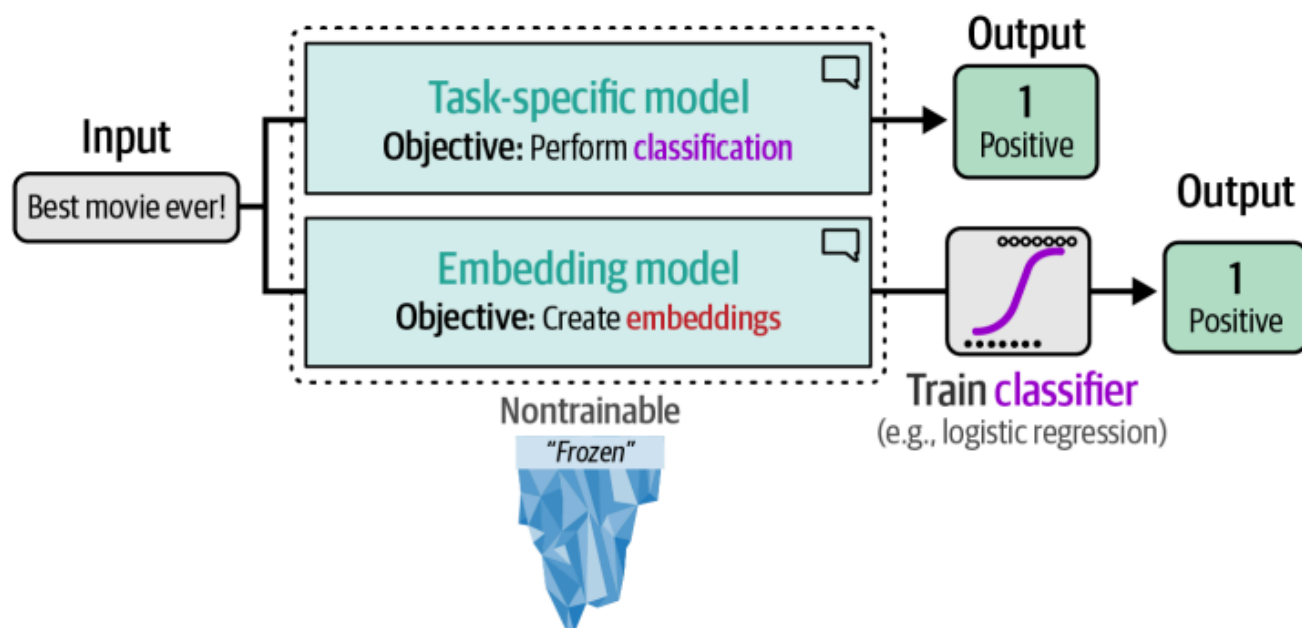
همان‌طور که در فصل قبل دیدیم، این مدل‌ها با تنظیم دقیق (Fine-Tuning) یک مدل پایه (Foundation Model) مانند BERT - بر روی یک وظیفه‌ی مشخص پایین‌دستی (Downstream Task) ساخته می‌شوند (مطابق شکل زیر).



یک مدل خاص وظیفه (Task-Specific Model) در واقع نوعی مدل بازنمایی (Representation Model) است که برای یک کار مشخص، مثل تحلیل احساسات (Sentiment Analysis)، آموزش داده می‌شود. برای نمونه، BERT را می‌توان مدلی از این نوع دانست.

در مقابل، یک مدل امبدینگ (Embedding Model) بازنمایی‌هایی کلی و چندمنظوره (General-Purpose Embeddings) تولید می‌کند که تنها به طبقه‌بندی محدود نیستند و می‌توان آن‌ها را در زمینه‌های مختلفی به کار گرفت. یکی از این کاربردها جست‌وجوی معنایی (Semantic Search) است که در فصل‌های بعدی به آن پرداخته خواهد شد.

فرآیند تنظیم دقیق مدل **BERT (Fine-Tuning BERT)** برای طبقه‌بندی و همچنین نحوه ساخت یک مدل امبدینگ (**Embedding Model**) در فصل‌های بعدی توضیح داده می‌شود. در این فصل، هر دو مدل ثابت (غیرقابل آموزش (**Frozen / Nontrainable**)) نگه داشته می‌شوند و تنها از خروجی آن‌ها استفاده می‌کنیم.



در اینجا ما ابتدا از مدل‌های از پیش آموزش‌دیده (**Pretrained Models**) استفاده می‌کنیم که دیگران قبلاً برای ما تنظیم دقیق (**Fine-Tuned**) کرده‌اند و بررسی خواهیم کرد چگونه می‌توان از آن‌ها برای طبقه‌بندی نقدهای فیلم انتخاب‌شده بهره برد.

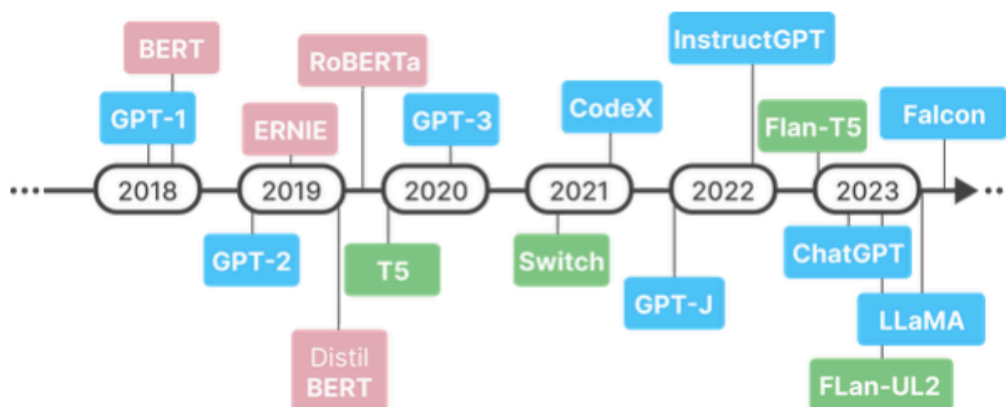
انتخاب مدل

انتخاب مدل مناسب همیشه کار ساده‌ای نیست، به‌خصوص وقتی در [Hugging Face Hub](https://huggingface.co) بیش از ۶۰,۰۰۰ مدل برای طبقه‌بندی متن (**Text Classification**) و بیش از ۸,۰۰۰ مدل که بردارهای امبدینگ (**Embeddings**) را تولید می‌کنند، موجود باشد. علاوه بر تعداد زیاد، مهم است مدلی را انتخاب کنید که با نیازهای شما هماهنگ باشد و در عین حال زبان، معماری، اندازه و عملکرد مناسبی داشته باشد.

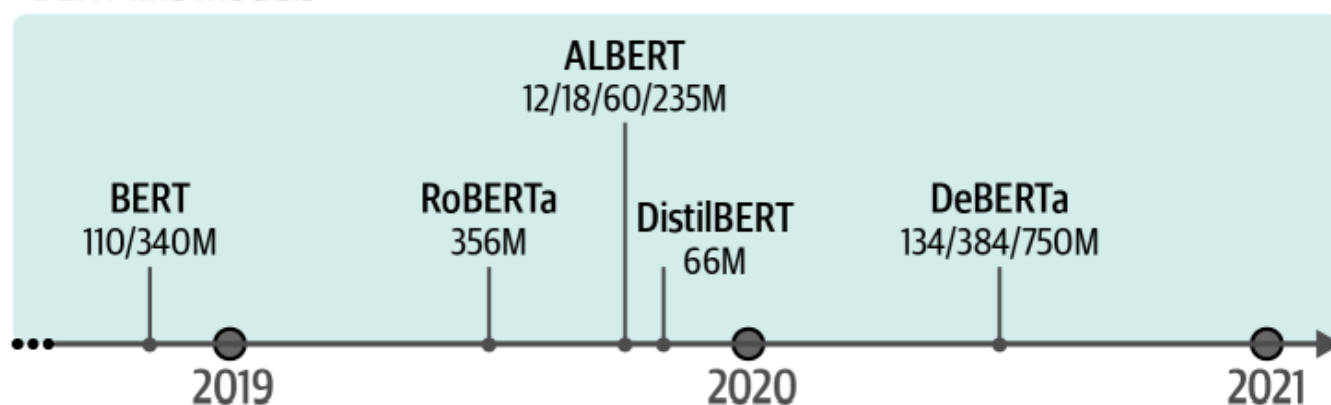
بیاید ابتدا به معماری مدل‌ها (**underlying architecture**) نگاه کنیم. همان‌طور که در فصل اول دیدیم، **BERT** که یک مدل فقط رمزگذار (**Encoder-Only Architecture**) است، گزینه‌ای محبوب برای ساخت مدل‌های خاص وظیفه (**Task-Specific Models**) و مدل‌های امبدینگ (**Embedding Models**) به شمار می‌رود. هرچند مدل‌های تولیدی (**Generative Models**) مانند خانواده **GPT** بسیار قدرتمند هستند، مدل‌های فقط رمزگذار در انجام وظایف مشخص هم عملکرد بسیار خوبی دارند و معمولاً حجم کمتری دارند.

در طول سال‌ها، نسخه‌های مختلفی از **BERT** ساخته شده است، مثل **RoBERTa**، **DistilBERT**، **ALBERT** و **DeBERTa** که هرکدام در زمینه‌های مختلف آموزش دیده‌اند. در شکل زیر می‌توانید مروری بر برخی از این

مدل‌های شناخته‌شده داشته باشید.



BERT-like models



انتخاب مدل مناسب برای یک وظیفه می‌تواند خود یک هنر باشد. امتحان کردن هزاران مدل از پیش آموزش‌دیده که در **Hugging Face Hub** موجود هستند، عملی نیست؛ بنابراین باید در انتخاب مدل‌ها **هوشمندانه و بهینه** عمل کنیم. با این حال، چند مدل وجود دارند که **نقطه شروع خوبی** محسوب می‌شوند و دیدی از **عملکرد پایه** این نوع مدل‌ها به ما می‌دهند. می‌توان آن‌ها را به عنوان **مبنای قابل اعتماد** در نظر گرفت:

- [BERT base model \(uncased\)](#)
- [RoBERTa base model](#)
- [DistilBERT base model \(uncased\)](#)
- [DeBERTa base model](#)
- [bert-tiny](#)
- [ALBERT base v2](#)

برای **مدل خاص وظیفه (Task-Specific Model)** ما از **Twitter-RoBERTa-base** برای **تحلیل احساسات (Sentiment Analysis)** استفاده می‌کنیم. این مدل، نسخه‌ای از **RoBERTa** است که روی **توییت‌ها** برای **تحلیل احساسات تنظیم دقیق (Fine-Tuned)** شده است. اگرچه این مدل به‌طور خاص برای **نقد‌های فیلم** آموزش داده نشده، بررسی نحوه **توانایی تعمیم آن (Generalization)** می‌تواند جالب باشد.

برای انتخاب از بین مدل‌هایی که **بردارهای امبدینگ (Embeddings)** تولید می‌کنند، لیست **رتبه‌بندی MTEB** [MTEB leaderboard](#) نقطه شروع خوبی است. این لیست شامل مدل‌های **منبع باز و منبع بسته** است که

در چندین وظیفه مختلف **سنجش عملکرد (Benchmarking)** شده‌اند. هنگام انتخاب مدل، توجه داشته باشید که علاوه بر عملکرد مدل، سرعت استنتاج (Inference Speed) نیز در راه‌حل‌های عملی اهمیت بالایی دارد.

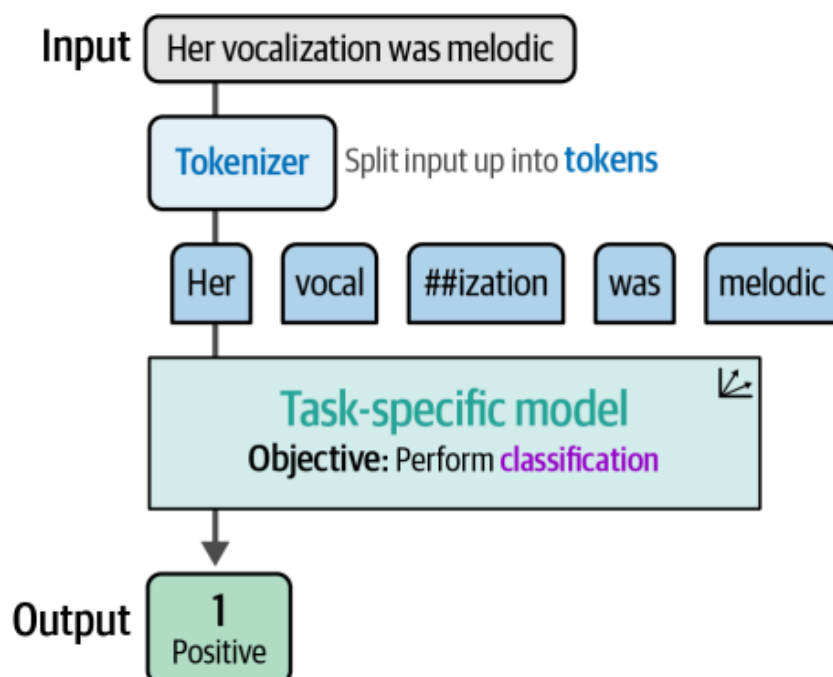
بنابراین، در این بخش از مدل `sentence-transformers/all-mpnet-base-v2`  به عنوان مدل **امبدینگ** استفاده می‌کنیم. این مدل **کوچک اما با عملکرد بالا** است و گزینه‌ای مناسب برای ما محسوب می‌شود.

استفاده از مدل خاص وظیفه

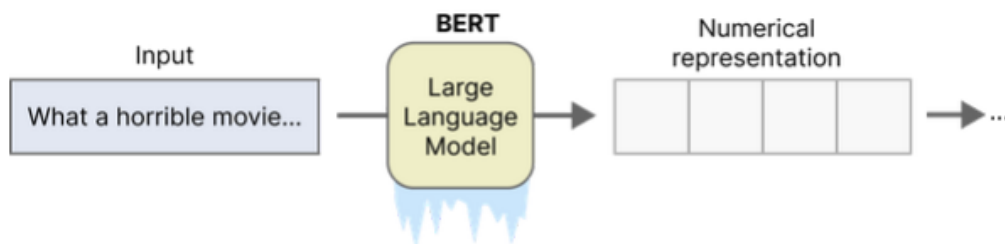
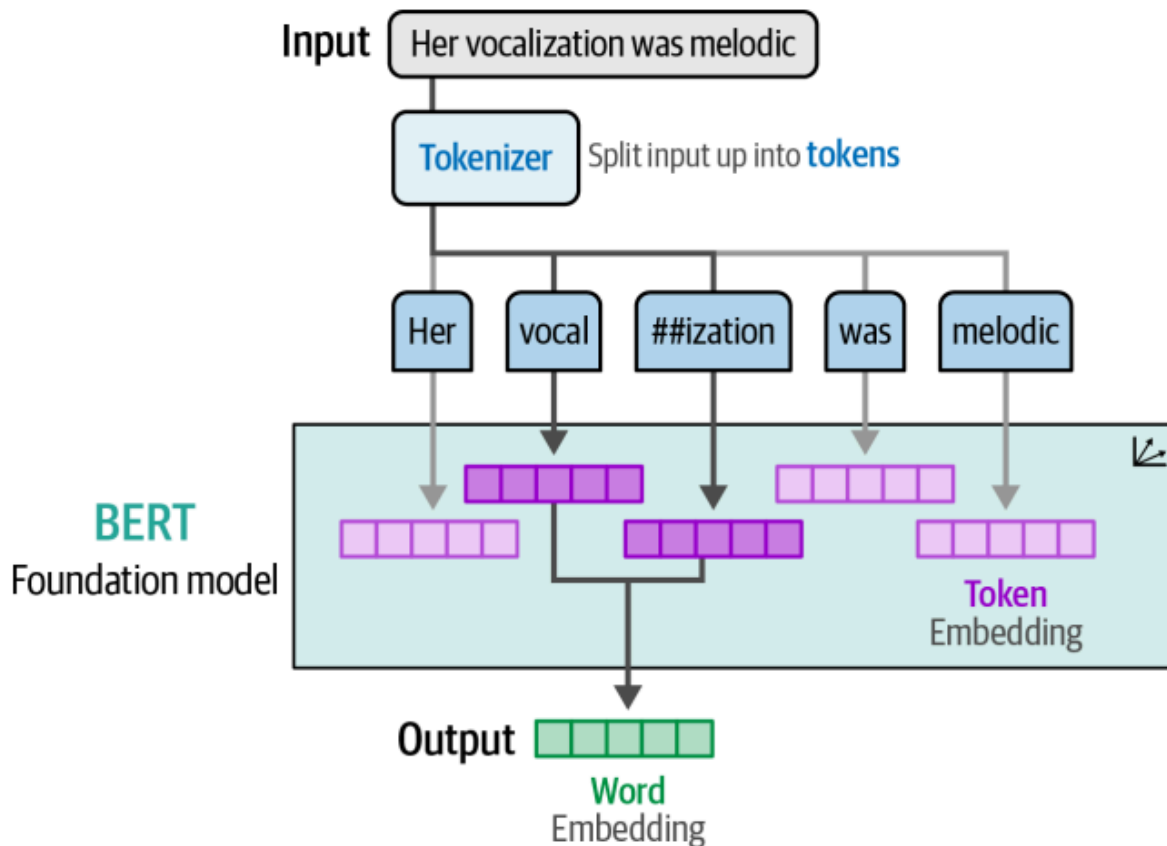
اکنون که مدل **بازنمایی خاص وظیفه (Task-Specific Representation Model)** خود را انتخاب کرده‌ایم، بیایید کار را با **بارگذاری مدل** آغاز کنیم:

```
from transformers import pipeline
# Path to our HF model
model_path = "cardiffnlp/twitter-roberta-base-sentiment-latest"
# Load model into pipeline
pipe = pipeline(
    model=model_path,
    tokenizer=model_path,
    return_all_scores=True,
    device="cuda:0"
)
```

هنگام بارگذاری مدل، توکنایزر نیز بارگیری می‌شود که مسئول تبدیل متن ورودی به توکن‌های مجزا است، همان‌طور که در شکل زیر نشان داده شده است. اگرچه مشخص کردن این پارامتر الزامی نیست، زیرا به‌طور خودکار بارگیری می‌شود، اما نشان می‌دهد که در پس‌زمینه چه اتفاقی رخ می‌دهد.



توکن‌ها هسته‌ی اصلی بیشتر مدل‌های زبانی هستند. همان‌طور که در شکل زیر نشان داده شده است، یکی از مزایای کلیدی این توکن‌ها این است که حتی اگر یک کلمه در داده‌های آموزشی نبوده باشد، مدل همچنان می‌تواند با شکستن آن به توکن‌های کوچک‌تر، بازنمایی مناسبی تولید کند.



پس از بارگذاری تمام اجزای لازم، می‌توانیم از مدل خود روی مجموعه‌ی تست استفاده کنیم:

```
import numpy as np
from tqdm import tqdm
from transformers.pipelines.pt_utils import KeyDataset
# Run inference
y_pred = []
for output in tqdm(pipe(KeyDataset(data["test"], "text")),
total=len(data["test"])):
    negative_score = output[0]["score"]
    positive_score = output[2]["score"]
```

```
assignment = np.argmax([negative_score, positive_score])
y_pred.append(assignment)
```

حال که پیش‌بینی‌های خود را تولید کرده‌ایم، تنها مرحله‌ی باقی‌مانده، ارزیابی آن‌ها است. تابعی ساده تعریف می‌کنیم که بتوانیم در سراسر این فصل از آن استفاده کنیم:

```
from sklearn.metrics import classification_report
def evaluate_performance(y_true, y_pred):
    """Create and print the classification report"""
    performance = classification_report(
        y_true, y_pred,
        target_names=["Negative Review", "Positive Review"]
    )
    print(performance)
```

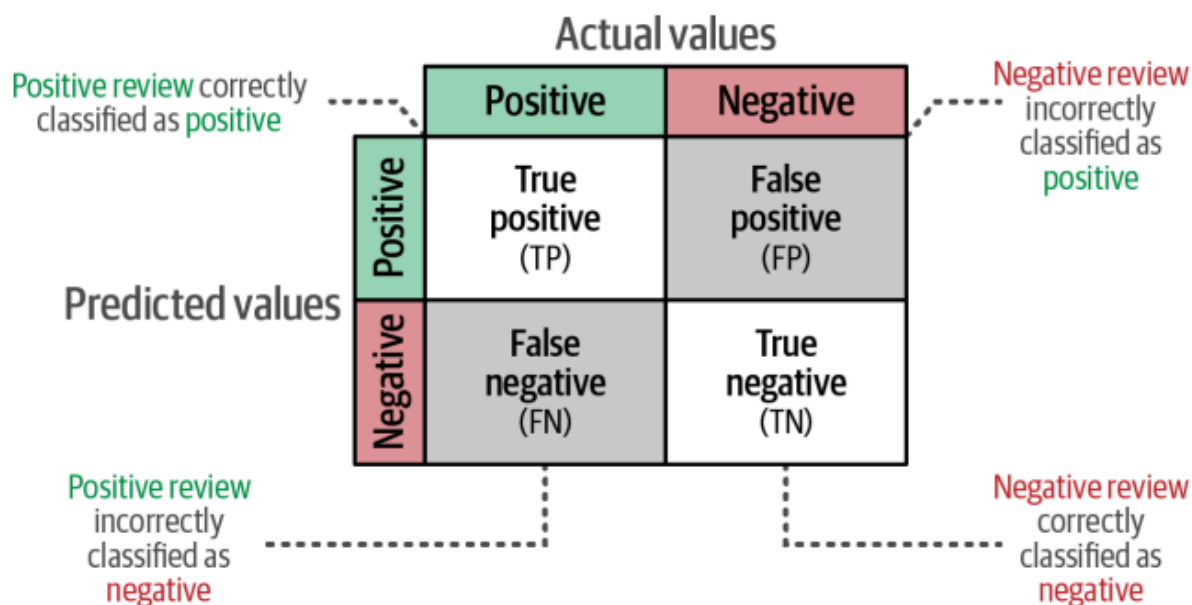
حال گزارش ارزیابی مدل را ایجاد می‌کنیم:

```
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.76	0.88	0.81	533
Positive Review	0.86	0.72	0.78	533
accuracy			0.80	1066
macro avg	0.81	0.80	0.80	1066
weighted avg	0.81	0.80	0.80	1066

برای درک بهتر گزارش ارزیابی، ابتدا بررسی می‌کنیم که چگونه می‌توان پیش‌بینی‌های درست و نادرست را تشخیص داد.

چهار حالت مختلف برای پیش‌بینی‌ها وجود دارد که در جدول زیر آورده شده است و به آن ماتریس اغتشاش (Confusion Matrix) می‌گویند.



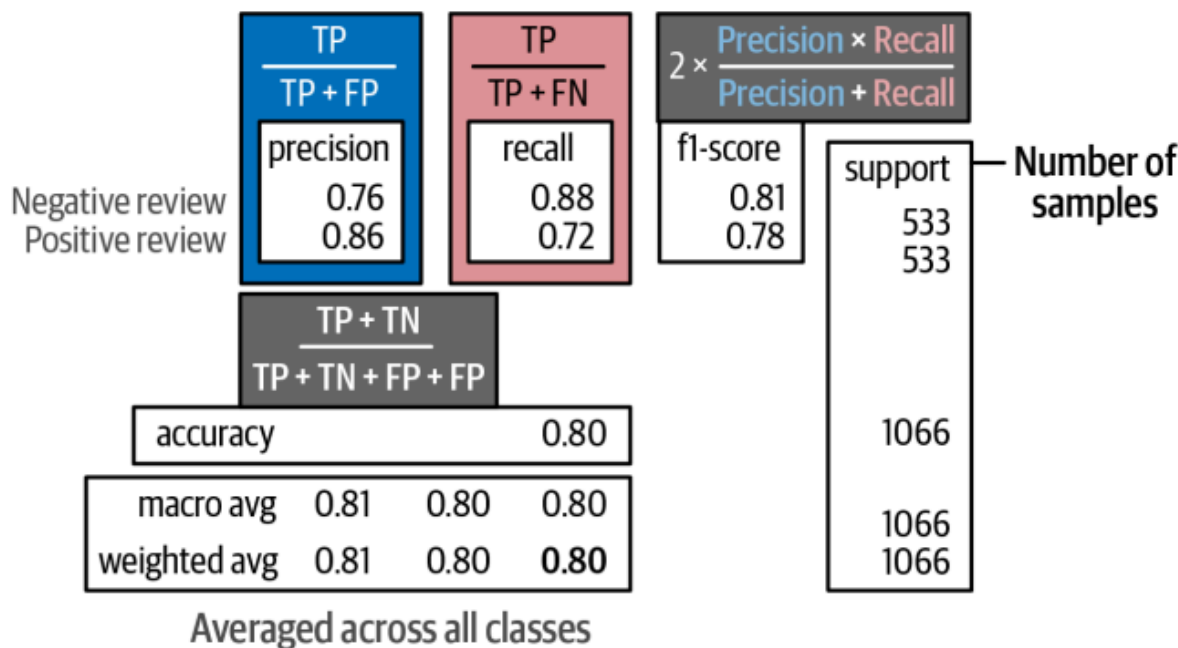
	پیش‌بینی مثبت	پیش‌بینی منفی
کلاس واقعی مثبت	True Positive (TP)	False Negative (FN)
کلاس واقعی منفی	False Positive (FP)	True Negative (TN)

ماتریس اغتشاش که چهار نوع پیش‌بینی ممکن را نشان می‌دهد.

با استفاده از ماتریس اغتشاش، چندین معیار ارزیابی قابل استخراج است:

- **دقت (Precision):** نشان می‌دهد که چه درصدی از نمونه‌های پیش‌بینی‌شده‌ی مثبت واقعاً مثبت هستند.
- **بازخوانی (Recall):** نشان می‌دهد که چه درصدی از نمونه‌های مثبت واقعی توسط مدل به‌درستی شناسایی شده‌اند.
- **دقت کلی (Accuracy):** درصد کل پیش‌بینی‌های صحیح را نمایش می‌دهد.
- **امتیاز F1:** میانگینی از دقت و بازخوانی است و شاخصی از عملکرد کلی مدل ارائه می‌دهد.

این معیارها در شکل زیر نشان داده شده‌اند.



در این مثال، از میانگین وزنی (Weighted Average) امتیاز F1 استفاده می‌کنیم، زیرا این مقدار تضمین می‌کند که همه‌ی کلاس‌ها به یک اندازه مهم در نظر گرفته شوند. مدل BERT از پیش‌آموزش‌دیده‌ی ما، امتیاز F1 برابر ۰.۸۰ را ارائه می‌دهد که برای مدلی که به‌طور خاص روی داده‌های دامنه‌ی ما آموزش ندیده است، عملکرد خوبی محسوب می‌شود.

برای بهبود عملکرد مدل، می‌توان چندین رویکرد اتخاذ کرد، از جمله انتخاب مدلی که به‌طور خاص بر داده‌های دامنه‌ی ما (مثلاً نقدهای فیلم) آموزش دیده است، مانند مدل [DistilBERT base uncased finetuned](#) [SST-2](#). همچنین، می‌توانیم تمرکز خود را به نوع دیگری از مدل‌های بازنمایی، یعنی مدل‌های امبدینگ، تغییر دهیم.

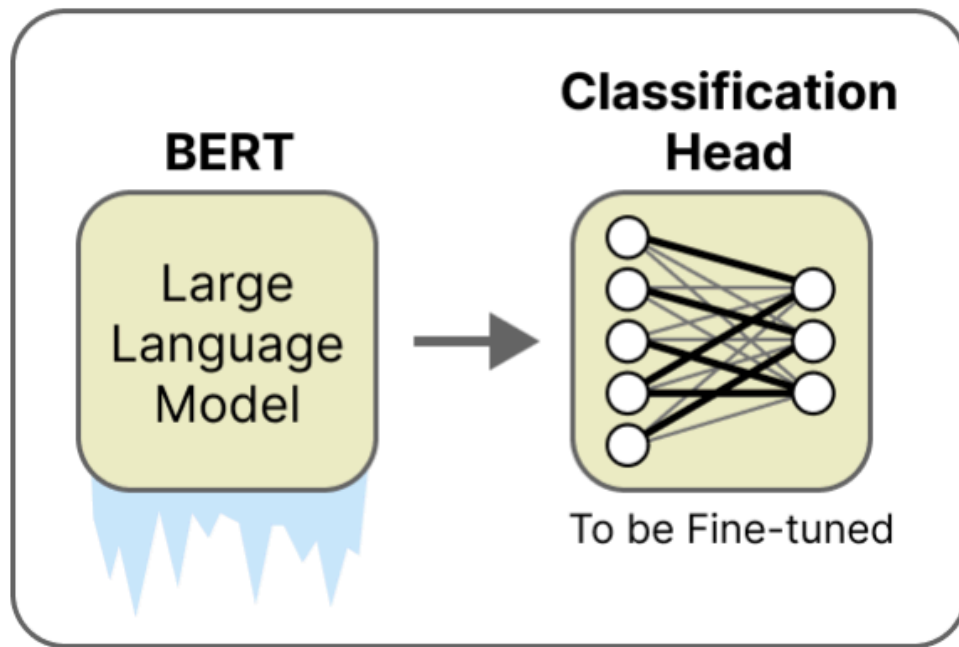
وظایف طبقه‌بندی که از بازنمایی‌ها استفاده می‌کنند

در مثال قبلی، ما از یک مدل پیش‌آموزش‌دیده خاص وظیفه برای تحلیل احساسات استفاده کردیم. اما پرسش اینجاست: اگر مدلی آماده برای چنین وظیفه‌ای در دسترس نباشد، آیا ناچاریم خود یک مدل نمایشی را به‌طور جداگانه تنظیم دقیق (Fine-Tune) کنیم؟ پاسخ روشن است: خیر.

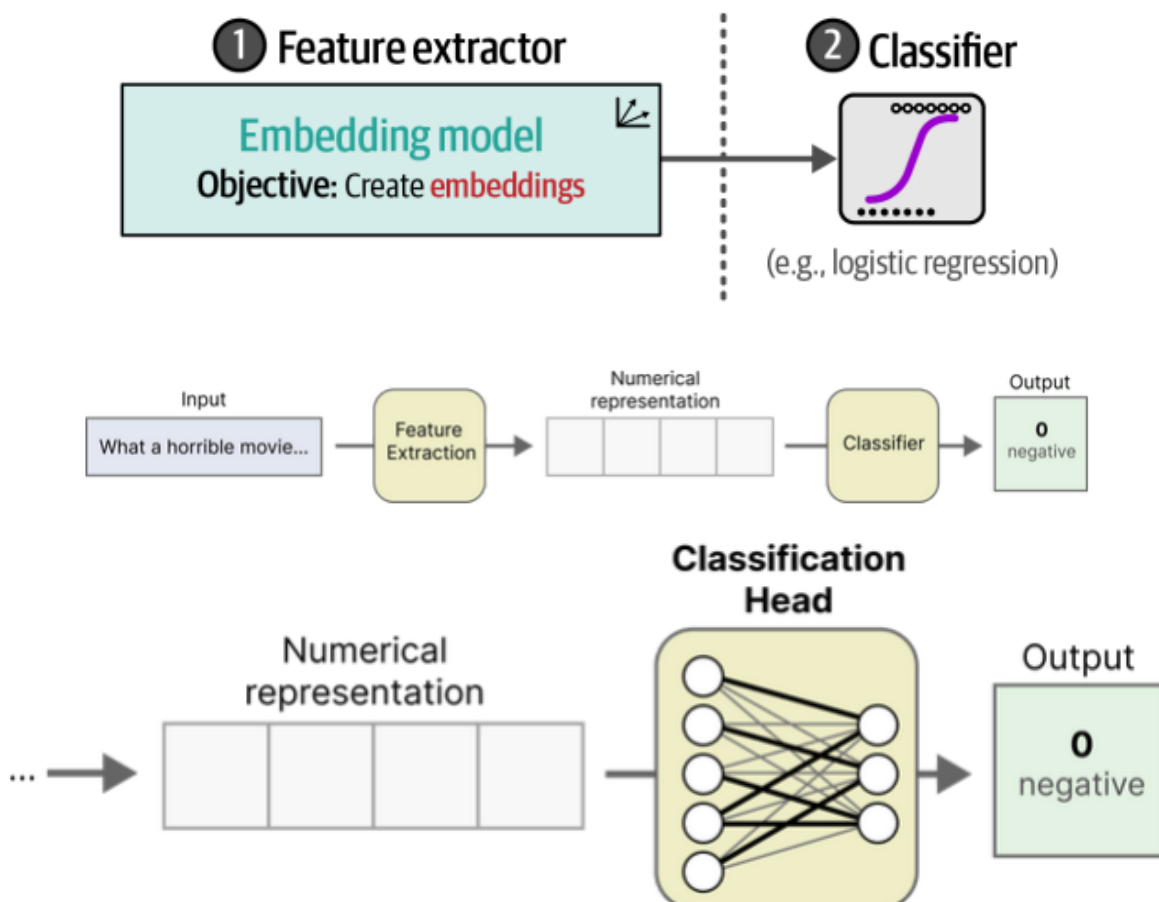
در برخی موارد، اگر منابع محاسباتی کافی در اختیار داشته باشید (مطابق فصل‌های آتی)، می‌توانید مدل را شخصاً تنظیم دقیق (Fine-Tune) کنید. اما از آنجا که چنین امکاناتی همیشه در دسترس همه نیست، اینجاست که مدل‌های امبدینگ چندمنظوره (General-Purpose Embedding Models) به کار می‌آیند و راه‌حلی کارآمد ارائه می‌دهند.

طبقه‌بندی نظارت‌شده

Feature Extractor + Classifier

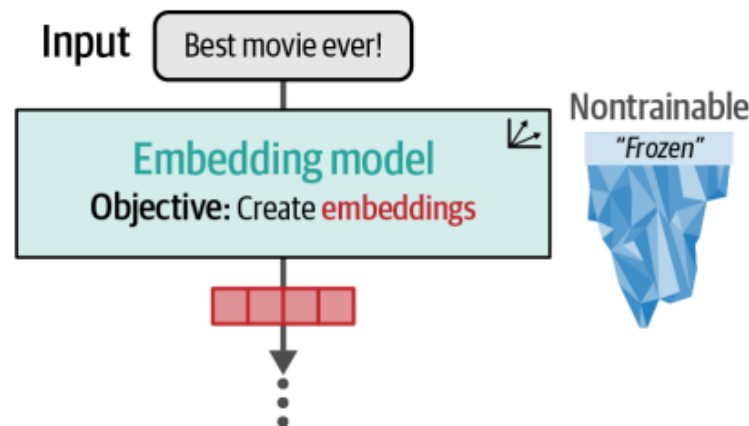


برخلاف مثال قبلی، ما می‌توانیم بخشی از فرآیند آموزش را خودمان انجام دهیم و آن را از دیدگاهی کلاسیک‌تر مورد بررسی قرار دهیم. به جای استفاده مستقیم از مدل نمایشی برای طبقه‌بندی، ما از یک مدل امبدینگ (**Embedding Model**) برای استخراج ویژگی‌ها (**Features**) استفاده خواهیم کرد. سپس این ویژگی‌ها به یک طبقه‌بند (**Classifier**) داده می‌شوند و به این ترتیب، یک روش **دو مرحله‌ای** شکل می‌گیرد، همان‌طور که در شکل زیر دیده می‌شود.



یکی از مزایای اصلی این تقسیم‌بندی این است که نیازی به تنظیم دقیق مدل تعبیه (Embedding Model) نداریم، کاری که معمولاً هزینه‌بر است. در عوض، می‌توانیم یک طبقه‌بند ساده مانند رگرسیون لجستیک (Logistic Regression) را روی CPU آموزش دهیم.

در مرحله‌ی اول، ورودی متنی خود را با استفاده از مدل امبدینگ (Embedding Model) به بردارهای امبدینگ (Embeddings) تبدیل می‌کنیم، همان‌طور که در شکل زیر نشان داده شده است. توجه داشته باشید که این مدل نیز ثابت (Frozen) نگه داشته می‌شود و در طول فرآیند آموزش به‌روزرسانی نمی‌شود.



ما می‌توانیم این مرحله را با استفاده از sentence-transformers انجام دهیم که یک بسته محبوب برای استفاده از مدل‌های بازنمایی پیش‌آموزش‌دیده است.

ساخت بازنمایی‌ها ساده است:

```
from sentence_transformers import SentenceTransformer
# Load model
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")
# Convert text to embeddings
train_embeddings = model.encode(data["train"]["text"], show_progress_bar=True)
test_embeddings = model.encode(data["test"]["text"], show_progress_bar=True)
```

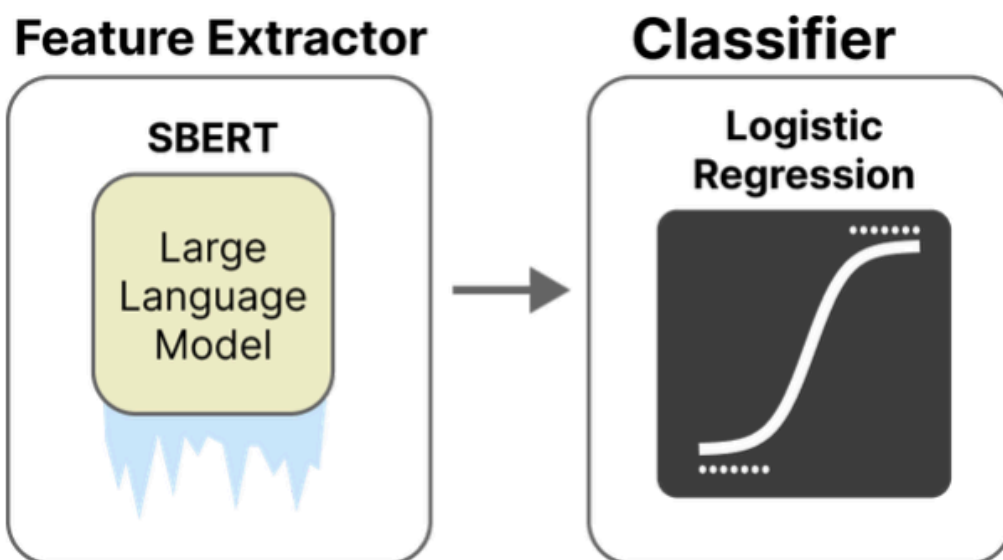
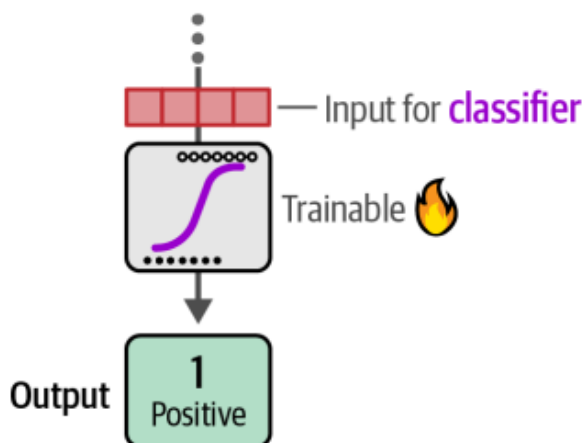
همان‌طور که در فصل اول دیدیم، این بردارهای امبدینگ (Embeddings) نشان‌دهنده‌های عددی (Numerical Representations) متن ورودی هستند. تعداد مقادیر یا همان ابعاد (Dimension) هر بردار بستگی به مدل امبدینگ پایه (Underlying Embedding Model) دارد. بیایید این موضوع را برای مدل خود بررسی کنیم:

```
train_embeddings.shape
```

```
(8530, 768)
```

این نشان می‌دهد که هر یک از 8530 سند ورودی ما ابعاد بازنمایی 768 دارند و بنابراین هر بازنمایی شامل 768 مقدار عددی است.

در مرحله دوم، این بردارهای امبدینگ (Embeddings) به عنوان ویژگی‌های ورودی (Input Features) به طبقه‌بند (Classifier) داده می‌شوند، همان‌طور که در شکل زیر نشان داده شده است. این طبقه‌بند قابل آموزش است و محدود به رگرسیون لجستیک (Logistic Regression) نیست؛ می‌تواند هر نوع مدل یا ساختاری که وظیفه‌ی طبقه‌بندی را انجام دهد داشته باشد.



ما این مرحله را به ساده‌ترین حالت ممکن در نظر گرفته و از رگرسیون لجستیک (Logistic Regression) به عنوان طبقه‌بند استفاده می‌کنیم. برای آموزش آن، کافی است از بردارهای امبدینگ تولیدشده (Generated Embeddings) همراه با برچسب‌های (Labels) داده‌ها استفاده کنیم:

```
from sklearn.linear_model import LogisticRegression
# Train a logistic regression on our train embeddings
clf = LogisticRegression(random_state=42)
clf.fit(train_embeddings, data["train"]["label"])
```

سپس بایبید مدل خود را ارزیابی کنیم:

```
# Predict previously unseen instances
y_pred = clf.predict(test_embeddings)
```

```
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.85	0.86	0.85	533
Positive Review	0.86	0.85	0.85	533
accuracy			0.85	1066
macro avg	0.85	0.85	0.85	1066
weighted avg	0.85	0.85	0.85	1066

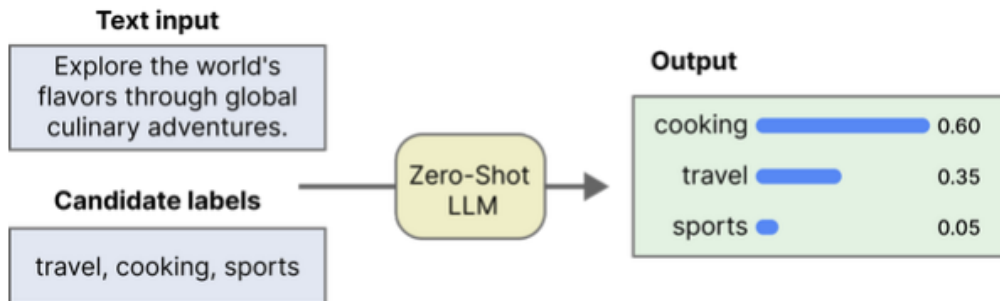
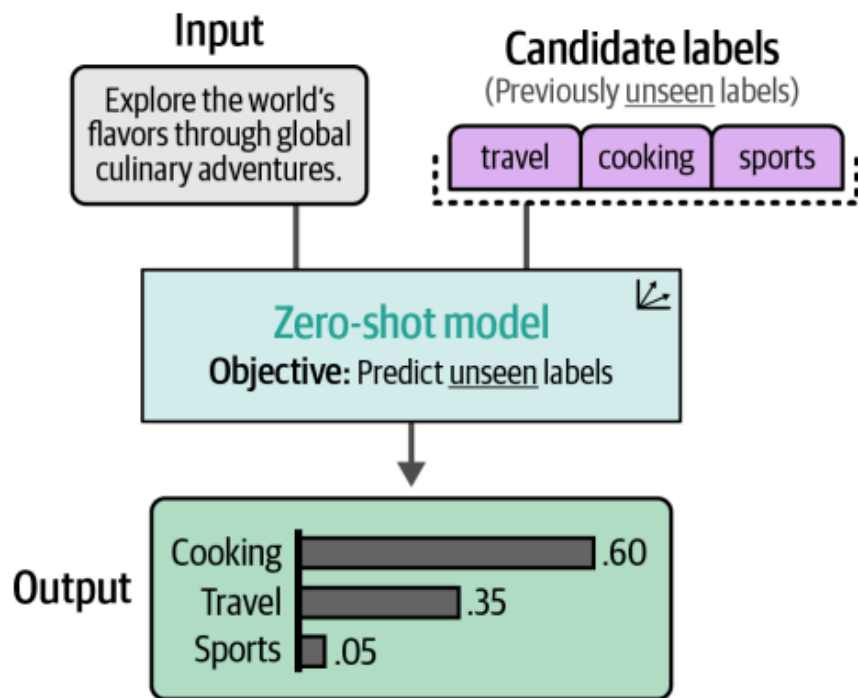
با آموزش یک طبقه‌بند روی بردارهای امبدینگ، توانستیم به امتیاز F1 برابر با ۰.۸۵ دست پیدا کنیم! این نتیجه نشان می‌دهد که چگونه می‌توان یک طبقه‌بند سبک و کم‌هزینه آموزش داد، در حالی که مدل تعبیه پایه همچنان ثابت و بدون تغییر (frozen) باقی می‌ماند.

در این مثال، برای استخراج بردارهای امبدینگ (Embeddings) از sentence-transformers استفاده کردیم که با بهره‌گیری از GPU می‌تواند سرعت استنتاج (Inference) را افزایش دهد. با این حال، می‌توان این وابستگی به GPU را با استفاده از یک API خارجی برای تولید این بردارها از بین برد. از جمله گزینه‌های محبوب برای این کار، سرویس‌های Cohere و OpenAI هستند. به این ترتیب، کل روند پردازش (Pipeline) می‌تواند تنها روی CPU اجرا شود.

اگر داده‌های برچسب‌خورده نداشته باشیم، چه کار باید کنیم؟

در مثال قبلی، داده‌های برچسب‌گذاری‌شده داشتیم که می‌توانستیم از آن‌ها استفاده کنیم، اما در عمل همیشه چنین داده‌هایی در دسترس نیستند. جمع‌آوری این داده‌ها اغلب زمان‌بر و پرهزینه است و نیاز به نیروی انسانی زیاد دارد. علاوه بر این، این سوال پیش می‌آید که آیا جمع‌آوری این برچسب‌ها واقعاً ارزشش را دارد؟

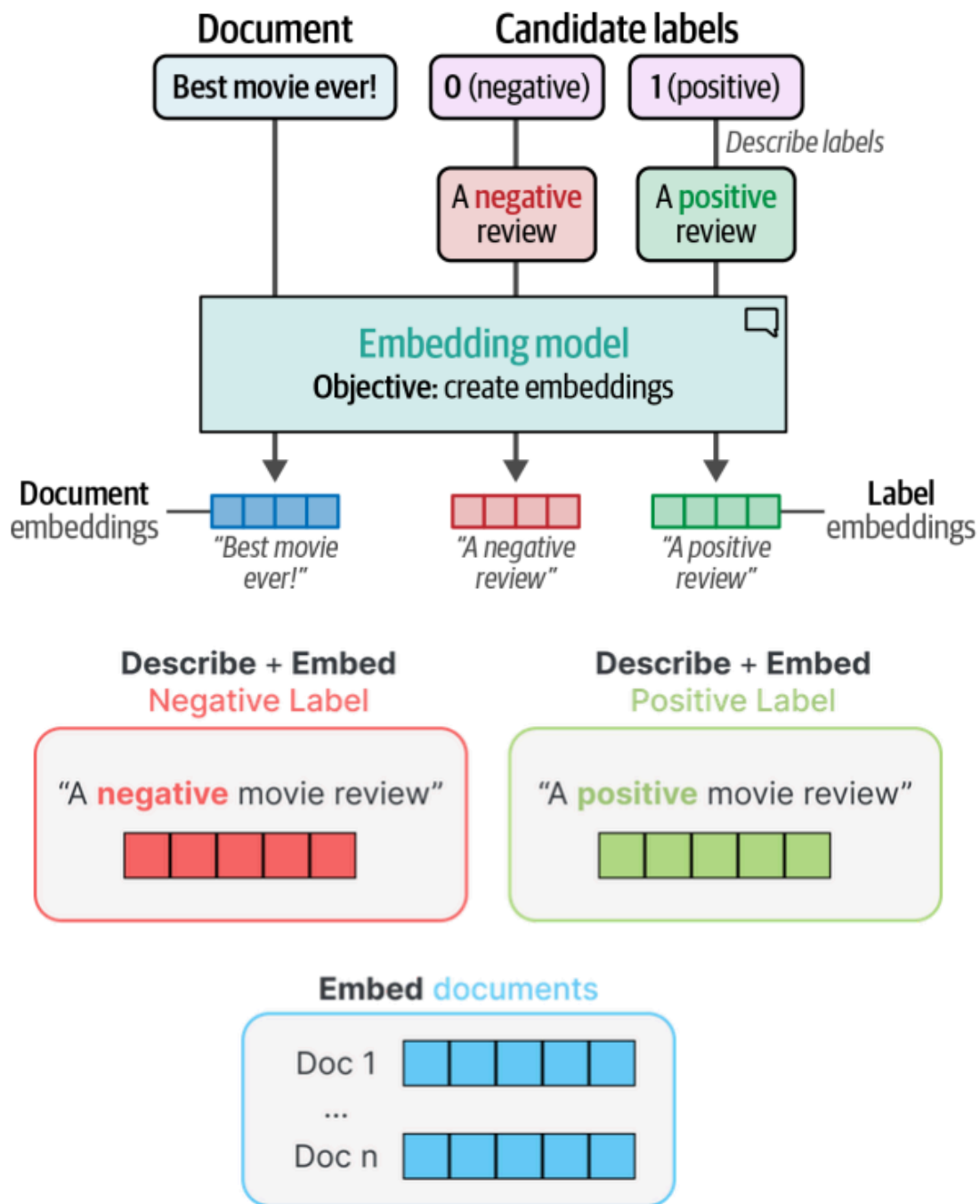
برای بررسی این موضوع، می‌توانیم از روش طبقه‌بندی زیرو-شات (Zero-Shot Classification) استفاده کنیم. در این روش، بدون هیچ داده آموزشی برچسب‌خورده، مدل تلاش می‌کند تشخیص دهد که متن ورودی به کدام دسته تعلق دارد. به عبارت دیگر، ما تنها نام برچسب‌ها را داریم، اما هیچ نمونه‌ای از داده‌های آموزشی برای آن‌ها موجود نیست. هدف این روش این است که مدل بتواند برچسب مناسب هر متن را پیش‌بینی کند، حتی اگر قبلاً روی آن برچسب‌ها آموزش ندیده باشد.



در **طبقه‌بندی Zero-shot**، ما هیچ داده برچسب‌خورده‌ای در اختیار نداریم. به جای آن، می‌توانیم هر برچسب را با یک توضیح متنی مشخص کنیم که معنای آن را روشن سازد. برای مثال، برای برچسب «منفی» در نقدهای فیلم می‌توان نوشت: «این یک نقد منفی برای فیلم است.»

سپس هم متن‌ها و هم توضیحات برچسب‌ها را به بردارهای امبدینگ عددی (**Embeddings**) تبدیل می‌کنیم. این کار به مدل امکان می‌دهد تا ارتباط متن با هر برچسب را تحلیل کند و پیش‌بینی مناسبی ارائه دهد، حتی بدون اینکه قبلاً نمونه‌ای از آن برچسب‌ها دیده باشد.

به این ترتیب، می‌توانیم بدون نیاز به داده‌های برچسب‌خورده واقعی، برچسب‌های هدف خود را تعریف و از آن‌ها برای طبقه‌بندی متن‌ها استفاده کنیم.

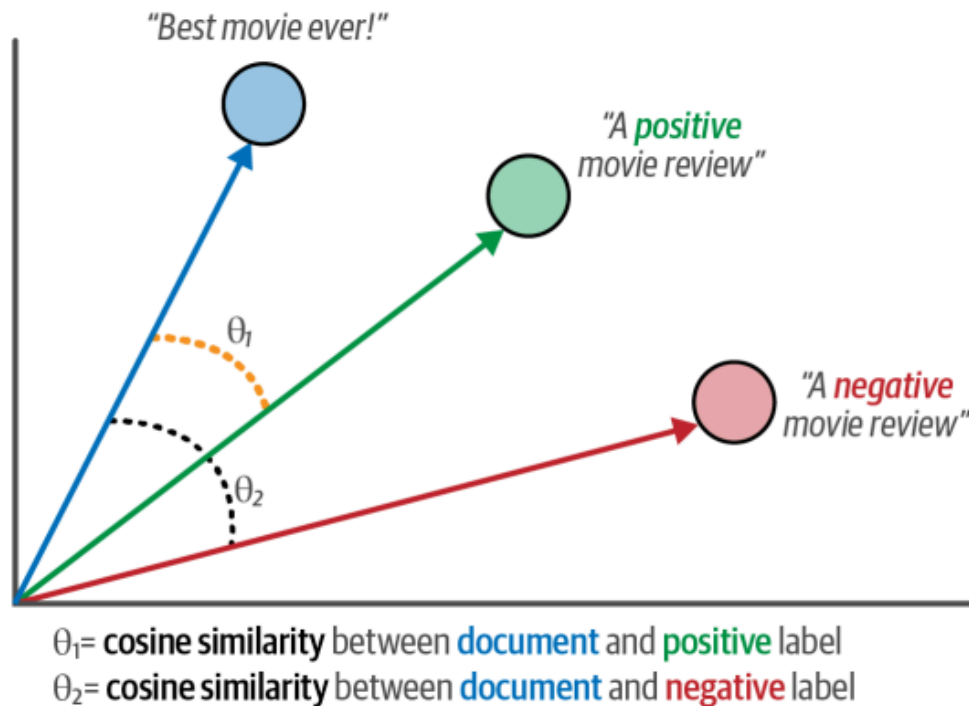


ما می‌توانیم بردارهای امبدینگ برچسب‌ها را با استفاده از تابع `encode` ایجاد کنیم، همان‌طور که قبلاً انجام دادیم:

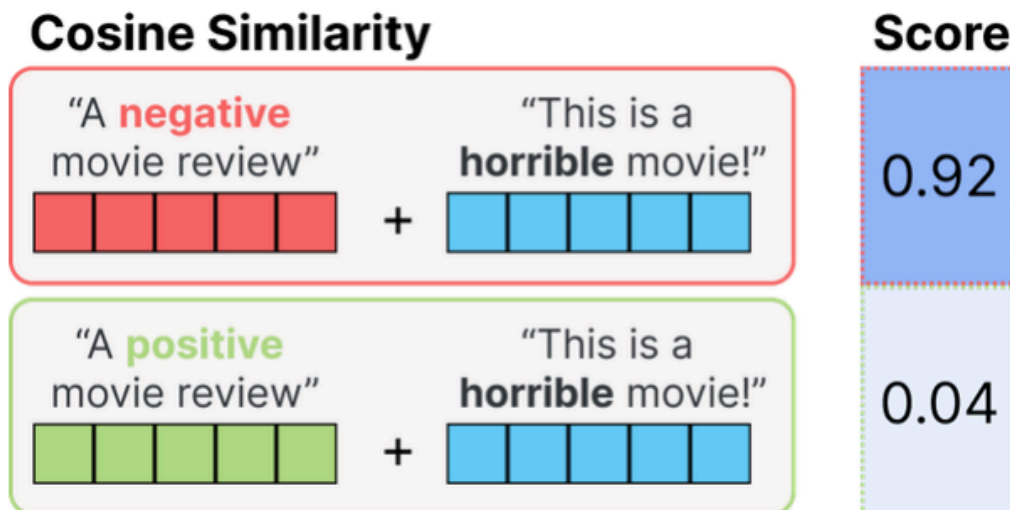
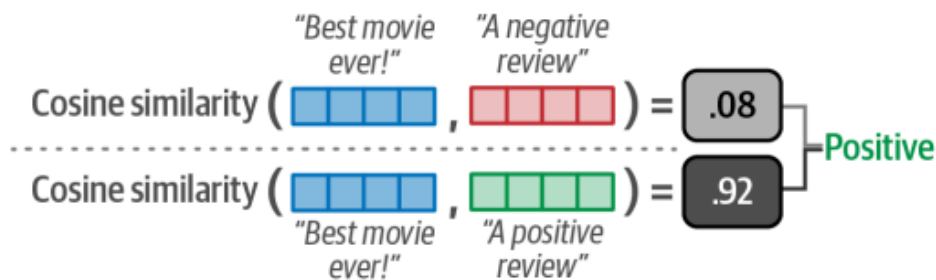
```
# Create embeddings for our labels
label_embeddings = model.encode(["A negative review", "A positive review"])
```

برای **اختصاص برچسب‌ها به متون**، می‌توانیم از **شباهت کسینوسی** بین هر **جفت متن و برچسب** استفاده کنیم. این مقدار، کسینوس زاویه بین دو بردار است که از **ضرب داخلی بردارهای امبدینگ** به‌دست می‌آید و

سپس بر حاصل ضرب طول آن‌ها تقسیم می‌شود، همان‌طور که در شکل زیر نشان داده شده است.



می‌توانیم از شباهت کسینوسی برای بررسی میزان تشابه یک متن با توضیحات برچسب‌های ممکن استفاده کنیم. برچسبی که بیشترین شباهت را با متن داشته باشد، به عنوان برچسب نهایی انتخاب می‌شود، همان‌طور که در شکل زیر نشان داده شده است.



برای محاسبه شباهت کسینوسی میان بردارها، کافی است امبدینگ‌های متن را با امبدینگ‌های برچسب‌ها مقایسه کنیم و در نهایت، نزدیک‌ترین جفت‌ها را به عنوان نتیجه انتخاب نماییم.

```
from sklearn.metrics.pairwise import cosine_similarity
# Find the best matching label for each document
sim_matrix = cosine_similarity(test_embeddings, label_embeddings)
y_pred = np.argmax(sim_matrix, axis=1)
```

همین بود! تنها کاری که لازم بود انجام دهیم این بود که برای برچسب‌ها نام یا توضیحی مشخص کنیم تا بتوانیم وظایف طبقه‌بندی خود را انجام دهیم. اکنون بیایید ببینیم این روش چقدر در عمل مؤثر است.

```
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.78	0.77	0.78	533
Positive Review	0.77	0.79	0.78	533
accuracy			0.78	1066
macro avg	0.78	0.78	0.78	1066
weighted avg	0.78	0.78	0.78	1066

امتیاز F1 برابر با ۰.۷۸ است که واقعاً نتیجه خوبی است با توجه به این‌که از هیچ داده برچسب‌خورده‌ای استفاده نکردیم! این نشان می‌دهد که بردارهای بازنمایی امبدینگ چقدر کاربردی و مفید هستند، به‌ویژه اگر خلاقانه از آن‌ها استفاده کنیم.

بیایید این خلاقیت را بیشتر آزمایش کنیم. ما "یک نقد منفی/مثبت" را به‌عنوان نام برچسب‌هایمان انتخاب کرده‌ایم، اما می‌توانیم این را بهبود دهیم. به‌جای آن، می‌توانیم آن‌ها را دقیق‌تر و مرتبط‌تر با داده‌هایمان کنیم و از "یک نقد بسیار منفی/مثبت فیلم" استفاده کنیم. این کار باعث می‌شود که بازنمایی بهتر مشخص کند که این نقد مربوط به فیلم است و تمرکز بیشتری بر **شدت احساسات متن در دو برچسب** خواهند داشت. امتحان کنید و ببینید این تغییر چه تاثیری بر نتایج می‌گذارد.

اگر با [zero-shot classification](#) در مدل‌های مبتنی بر ترنسفورمر آشنا باشید، ممکن است این پرسش برایتان مطرح شود که چرا در اینجا به‌جای این مدل‌ها، از بازنمایی‌ها (امبدینگ‌ها) استفاده کرده‌ایم. گرچه مدل‌های استنتاج زبان طبیعی (NLI) در طبقه‌بندی زیرو-شات عملکردی خارق‌العاده دارند، اما این مثال نشان می‌دهد که بازنمایی‌ها تا چه اندازه انعطاف‌پذیر بوده و می‌توانند در طیف گسترده‌ای از وظایف به کار گرفته شوند.

همان‌طور که در سراسر این کتاب خواهید دید، بازنمایی‌ها در بسیاری از کاربردهای هوش مصنوعی زبانی حضور دارند و درحالی‌که اغلب کم‌اهمیت انگاشته می‌شوند، در واقع نقشی اساسی و حیاتی ایفا می‌کنند.

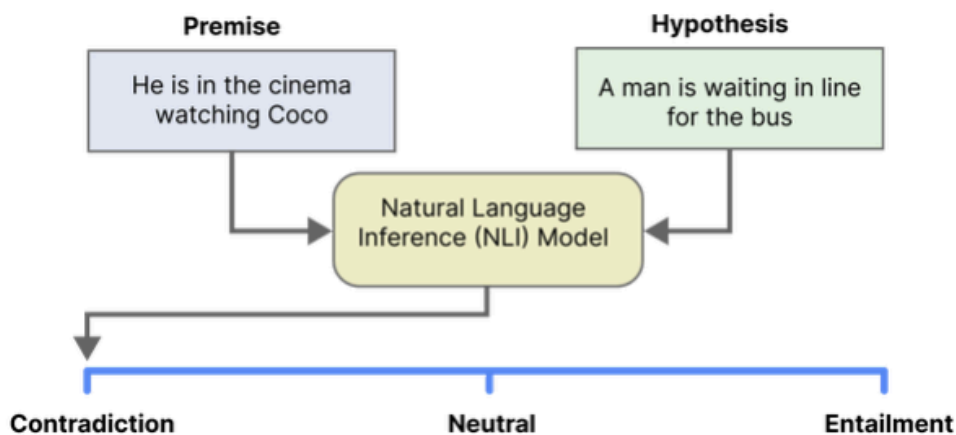


Figure 1-14. An example of natural language inference (NLI). The hypothesis is contradicted by the premise and is not relevant to one another.

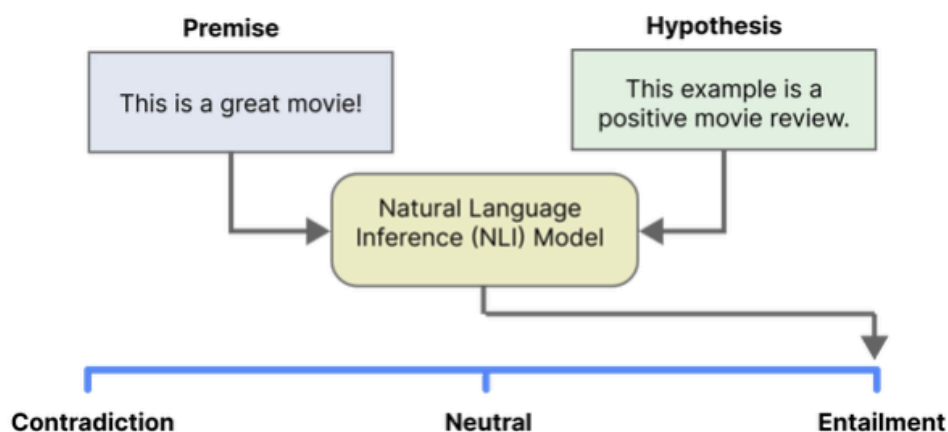
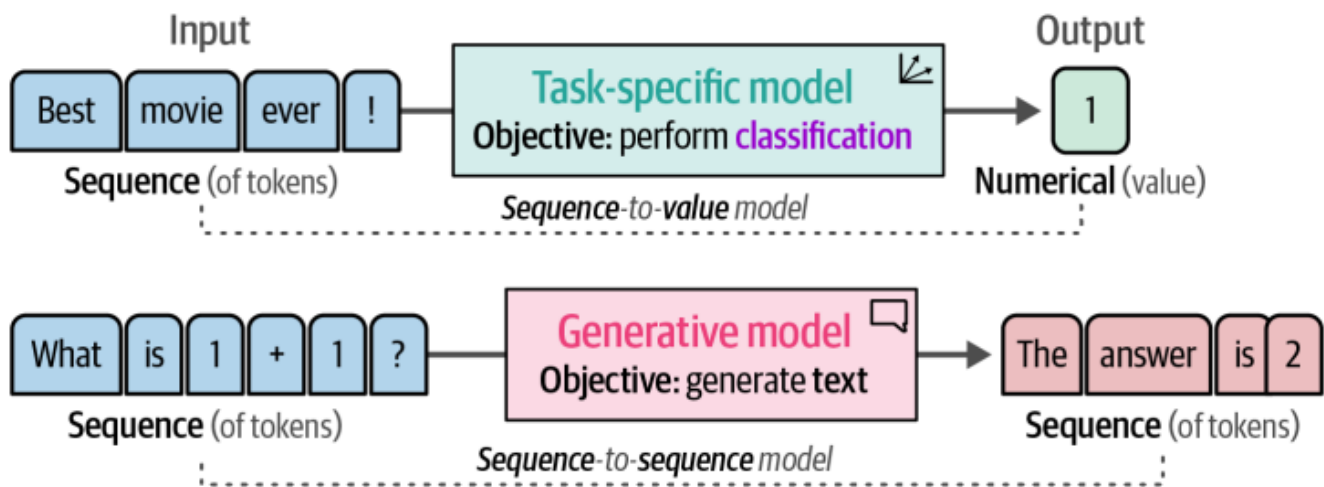


Figure 1-15. An example of zero-shot classification with natural language inference (NLI). The hypothesis is supported by the premise and the model will return that the review is indeed a positive movie review.

طبقه‌بندی متن با مدل‌های مولد (تولیدی)

طبقه‌بندی با مدل‌های زبانی تولیدی، مانند مدل‌های GPT شرکت OpenAI، کمی متفاوت از روش‌هایی است که تاکنون بررسی کردیم. این مدل‌ها متن ورودی را دریافت کرده و متن تولید می‌کنند و به همین دلیل به آن‌ها مدل‌های دنباله‌به‌دنباله (Sequence-to-Sequence) گفته می‌شود.

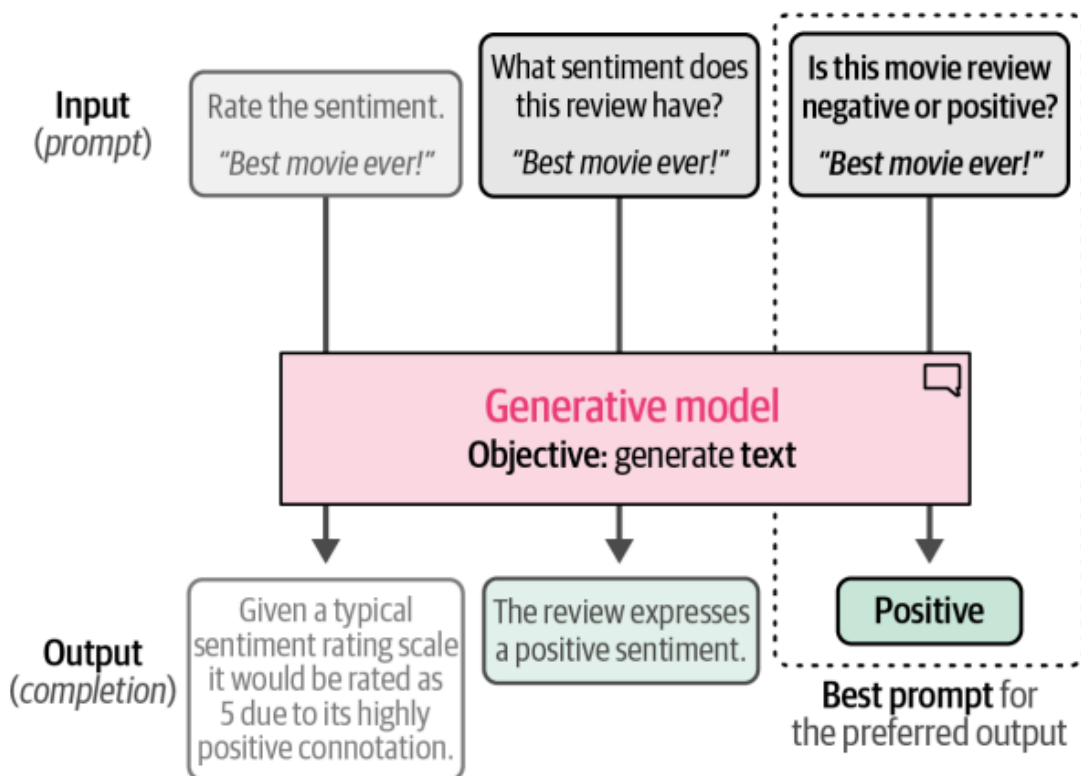
این رویکرد تفاوت قابل توجهی با مدل‌های خاص وظیفه دارد که مستقیماً یک دسته یا کلاس را به عنوان خروجی ارائه می‌دهند، همان‌طور که در شکل زیر نشان داده شده است.



این مدل‌های تولیدی معمولاً بر روی انواع متنوعی از وظایف آموزش دیده‌اند و به‌طور پیش‌فرض برای کاربرد خاص شما آماده نیستند. برای مثال، اگر یک نقد فیلم را بدون هیچ زمینه‌ای به مدل بدهیم، مدل نمی‌داند چه کاری باید انجام دهد.

به همین دلیل، لازم است زمینه و هدف خود را به مدل منتقل کنیم تا آن را به سمت پاسخ‌های مورد نظر هدایت کنیم. همان‌طور که در شکل ۱۸-۴ نشان داده شده است، این هدایت عمدتاً از طریق **دستورالعمل** یا **پرامپت (Prompt)** انجام می‌شود.

فرآیند بهبود تدریجی پرامپت برای رسیدن به خروجی دلخواه، به اصطلاح مهندسی پرامپت (**Prompt Engineering**) نامیده می‌شود.

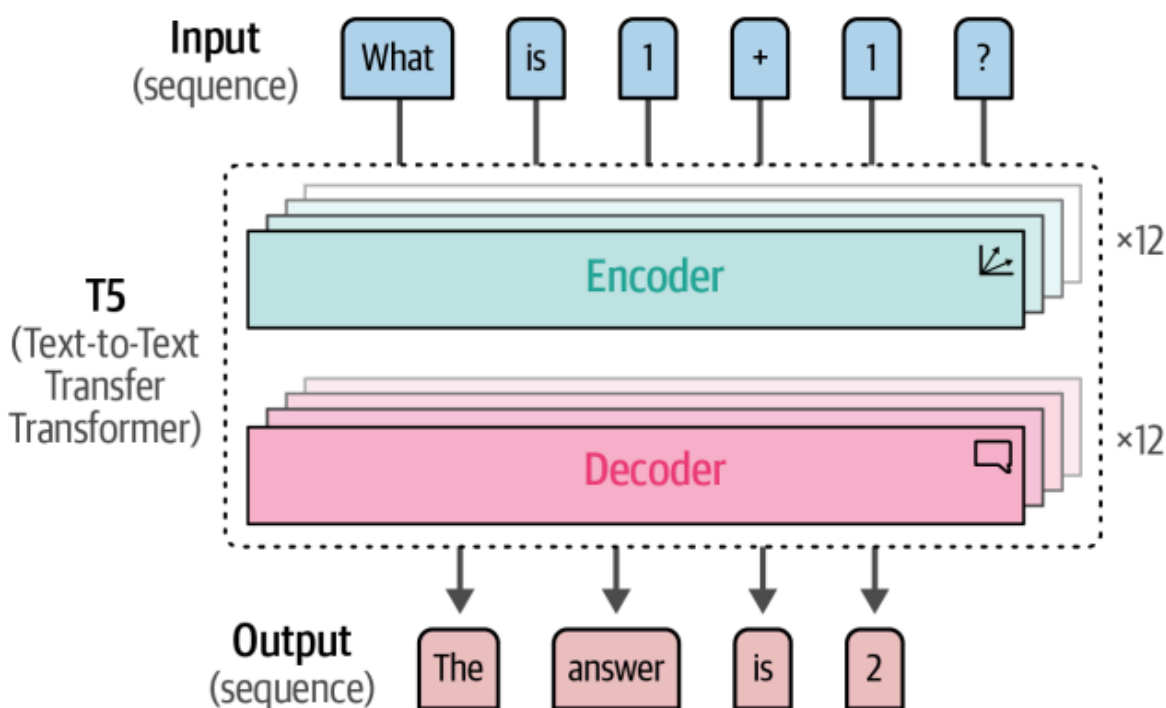


در این بخش، نشان خواهیم داد که چگونه می‌توانیم از انواع مختلف مدل‌های تولیدی برای انجام طبقه‌بندی حتی بدون دسترسی به مجموعه داده **Rotten Tomatoes**، استفاده کنیم.

به‌کارگیری مدل مبدل انتقال متن به متن (Text-to-Text Transfer Transformer) (T5) (T5)

در این کتاب تمرکز اصلی ما بر دو گروه از مدل‌ها خواهد بود: مدل‌های صرفاً رمزگذار (encoder-only) مانند BERT که برای بازنمایی داده‌ها به کار می‌روند، و مدل‌های صرفاً رمزگشا (decoder-only) مانند ChatGPT که در دسته‌ی مدل‌های مولد یا تولیدی قرار می‌گیرند. با این حال، همان‌طور که در فصل نخست اشاره شد، معماری اصلی ترنسفورمر در حقیقت بر پایه‌ی ترکیب رمزگذار-رمزگشا (encoder-decoder) شکل گرفته است. این دسته از مدل‌ها، درست مانند مدل‌های رمزگشا، در گروه مدل‌های دنباله‌به‌دنباله (Sequence-to-Sequence) جای می‌گیرند و برای تولید متن به کار می‌روند (عمدتاً ماهیتی مولد دارند).

یکی از مهم‌ترین خانواده‌های این معماری، مدل Text-to-Text Transfer Transformer یا T5 است. همان‌گونه که در شکل 4-19 دیده می‌شود، ساختار T5 شباهت زیادی به معماری اصلی ترنسفورمر دارد و از ۱۲ لایه رمزگذار و ۱۲ لایه رمزگشا تشکیل شده که به صورت متوالی (پشته‌ای) روی یکدیگر قرار گرفته‌اند.



مدل T5 یا همان Text-to-Text Transfer Transformer، یکی از مهم‌ترین دستاوردها در حوزه پردازش زبان طبیعی است. دلیل نام‌گذاری آن به «متن به متن» این است که این مدل، همه مسائل زبانی را در یک چارچوب واحد تعریف می‌کند: ورودی همیشه یک متن است و خروجی هم همیشه یک متن. به عنوان نمونه، اگر وظیفه ترجمه باشد، ورودی می‌تواند دستور «translate English to German: How are you?» باشد و خروجی متنی مانند «Wie geht es dir?» تولید شود. یا اگر وظیفه طبقه‌بندی احساسات باشد، ورودی جمله‌ای مثل «classify sentiment: I loved the movie» است و خروجی آن «positive» خواهد بود. حتی در پرسش و پاسخ، ورودی شامل متن پرسش و زمینه‌ی مربوطه است و خروجی پاسخ دقیق به همان پرسش خواهد بود.

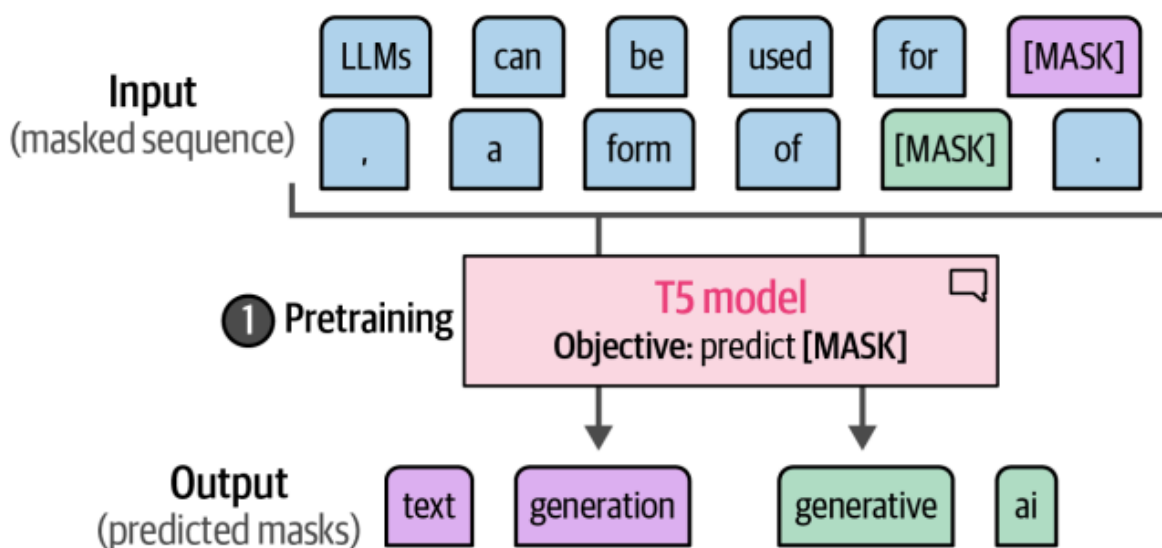
در مقایسه با دیگر مدل‌ها، BERT تنها نقش یک رمزگذار را ایفا می‌کند. این مدل متن ورودی را دریافت کرده و آن را به یک بردار یا بازنمایی عددی تبدیل می‌کند که بعداً می‌توان از آن در وظایفی مانند طبقه‌بندی یا خوشه‌بندی استفاده کرد. به بیان دیگر، BERT متن تولید نمی‌کند بلکه فقط معنا و ویژگی‌های آن را استخراج

می‌نماید. در مقابل، **ChatGPT** و مدل‌های مشابه آن، تنها بخش رمزگشا را دارند. این دسته از مدل‌ها پس از دریافت متن، به‌صورت پیوسته و زایشی متن جدید تولید می‌کنند. آن‌ها برای مکالمه و تولید متن آزاد بسیار توانمند هستند اما بیشتر وظایف را در قالب «پرسش-پاسخ» یا «دستور-پاسخ» انجام می‌دهند.

در این میان، جایگاه **T5** ویژه است، زیرا ترکیبی از رمزگذار و رمزگشا را همزمان به‌کار می‌گیرد. به همین دلیل، توانایی آن هم در فهم دقیق متن ورودی و هم در تولید متن خروجی بسیار بالاست. ویژگی منحصربه‌فرد T5 این است که با ساده‌سازی همه وظایف در قالب «تبدیل یک متن به متن دیگر»، انعطاف و کارآمدی بالایی در حل مسائل متنوع زبانی ارائه می‌دهد. به‌طور خلاصه می‌توان گفت: **BERT** مانند یک تحلیلگر است که معنا را استخراج می‌کند، **ChatGPT** همچون یک نویسنده است که متن تولید می‌کند و **T5** ترکیبی از هر دو نقش است؛ هم تحلیلگر و هم نویسنده، که همه چیز را در قالب متن به متن بازآفرینی می‌کند.

مراحل آموزش مدل T5

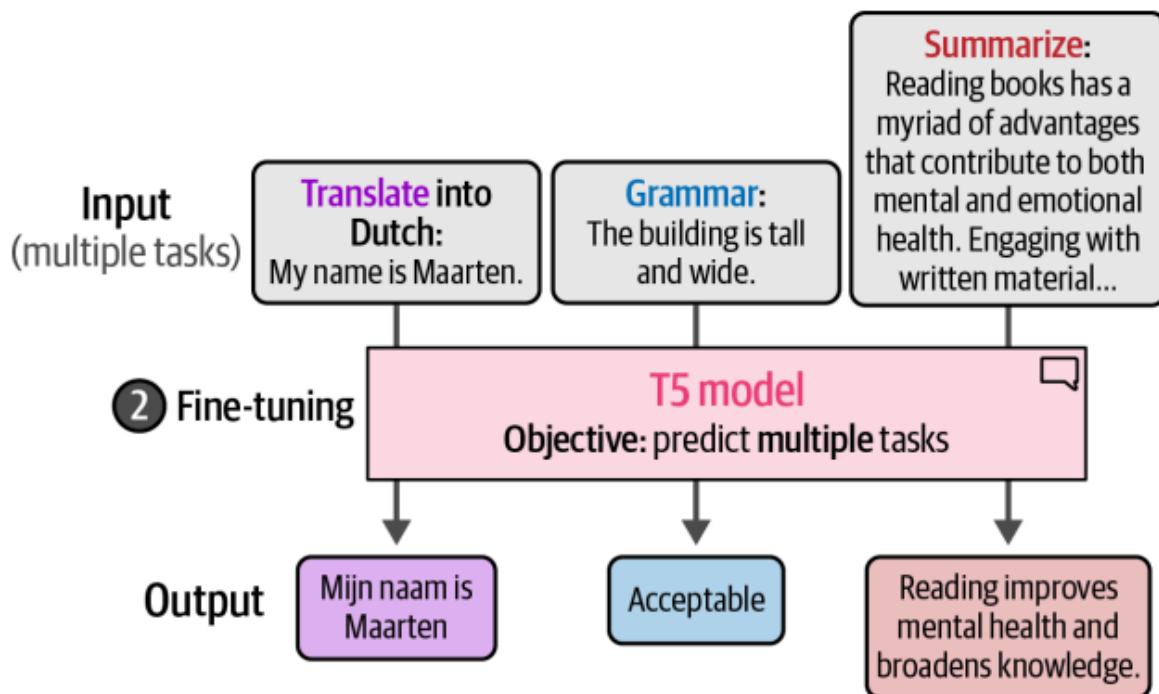
در معماری **T5**، مرحله‌ی پیش‌آموزش بر پایه‌ی روش **مدل‌سازی زبان با جای خالی (Masked Language Modeling)** انجام می‌شود. هدف این مرحله آن است که مدل بتواند بخش‌های **حذف‌شده‌ی متن** را پیش‌بینی کند. برخلاف روش‌های سنتی که تنها یک کلمه را مخفی می‌کردند، در T5 **چند کلمه یا یک عبارت متوالی** همزمان حذف می‌شوند. این شیوه موجب می‌شود مدل بتواند **ارتباط میان کلمات و ساختار جملات را به‌صورت عمیق‌تر درک کند** و توانایی تولید یا تکمیل متن را در سطحی طبیعی و منسجم به دست آورد. به بیان دیگر، مدل با این تمرین‌ها یاد می‌گیرد چگونه متن را حتی در شرایط ناقص بازسازی کند و درک زبانی خود را تقویت نماید.



در مرحله‌ی دوم آموزش، یعنی **تنظیم دقیق مدل پایه (Fine-Tuning)**، بخش اصلی قدرت و انعطاف‌پذیری مدل T5 آشکار می‌شود. برخلاف روش‌های سنتی که مدل را تنها برای یک **وظیفه‌ی مشخص** آموزش می‌دهند، در این رویکرد هر وظیفه فارغ از ماهیت دقیق آن (مانند ترجمه، طبقه‌بندی و تولید پاسخ) ابتدا به شکل یک **مسئله‌ی متن به متن (Sequence-to-Sequence)** در نظر گرفته می‌شود. به این ترتیب، مدل می‌تواند ورودی متنی را دریافت کرده و خروجی متنی مناسب تولید کند.

سپس، مدل به‌طور **همزمان روی چندین وظیفه مختلف** آموزش داده می‌شود، به گونه‌ای که یاد می‌گیرد چگونه ساختارهای زبانی مختلف را درک کند و پاسخ‌های مرتبط و دقیق تولید نماید. این روش باعث می‌شود مدل بتواند توانایی خود را **روی طیف گسترده‌ای از کاربردها** تقویت کند و قابلیت تعمیم بالاتری داشته باشد.

همان‌طور که در شکل زیر مشاهده می‌شود، این فرآیند یادگیری چندوظیفه‌ای، مدل را قادر می‌سازد تا به یک ابزار زبانی چندمنظوره و انعطاف‌پذیر تبدیل شود، نه صرفاً یک مدل تک‌کاربردی.



این روش **تنظیم دقیق مدل با دستورالعمل‌ها** در مقاله‌ی «Scaling Instruction-Finetuned Language Models» توسعه یافت. در این پژوهش، بیش از **هزار وظیفه‌ی متنوع** در فرآیند تنظیم دقیق اعمال شد، به‌گونه‌ای که مدل بتواند دستورالعمل‌ها را همانند آنچه در مدل‌های **GPT** مشاهده می‌کنیم، دنبال کند. نتیجه‌ی این رویکرد، ظهور خانواده‌ی مدل‌های **Flan-T5** بود که با بهره‌گیری از **تنوع گسترده‌ی وظایف**، توانایی تعمیم و انعطاف‌پذیری بیشتری پیدا کردند و قادرند در طیف وسیعی از کاربردهای پردازش زبان طبیعی عملکرد مؤثر و قابل اعتمادی ارائه دهند.

برای استفاده از مدل **Flan-T5** پیش‌آموزش‌دیده در وظیفه‌ی **طبقه‌بندی متن**، ابتدا آن را از طریق **وظیفه‌ی «تولید متن به متن» (text2text-generation)** بارگذاری می‌کنیم. این نوع وظیفه معمولاً برای مدل‌های **رمزگذار-رمزگشا (Encoder-Decoder)** طراحی شده و امکان می‌دهد مدل ورودی متنی را دریافت کرده و خروجی متنی مرتبط تولید کند.

```
# بارگذاری مدل
# Load our model
pipe = pipeline(
    "text2text-generation",
    model="google/flan-t5-small",
    device="cuda:0"
)
```

مدل Flan-T5 در اندازه‌های مختلفی از جمله small, base, large, xl, xxl ارائه شده است. در اینجا به منظور افزایش سرعت و سهولت اجرا، ما از **کوچک‌ترین نسخه (small)** استفاده می‌کنیم. با این حال، می‌توانید مدل‌های بزرگ‌تر را آزمایش کنید تا ببینید آیا نتایج بهتری حاصل می‌شود یا خیر.

برخلاف مدل‌های خاص وظیفه (Task-Specific)، در اینجا نمی‌توان صرفاً متن را به مدل داد و انتظار داشت که خودبه‌خود تمایلات یا احساسات متن را شناسایی کند. بلکه لازم است مدل را با دستورالعمل‌های مشخص (prompt) راهنمایی کنیم تا وظیفه‌ی موردنظر را انجام دهد.

بنابراین، برای هر سند، دستورالعمل (Prompt) زیر را به ابتدای هر متن اضافه می‌کنیم و آنرا در ستون T5 ذخیره می‌کنیم.

«آیا جمله‌ی زیر مثبت است یا منفی؟»

```
# آماده‌سازی داده‌ها
# Prepare our data
prompt = "Is the following sentence positive or negative? "
data = data.map(lambda example: {"t5": prompt + example['text']})
data
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 8530
  })
  validation: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 1066
  })
  test: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 1066
  })
})
```

پس از آماده‌سازی داده‌های، می‌توانیم روند پردازش و اجرای مدل را مانند کاری که در استفاده از مدل خاص وظیفه (Task-Specific) انجام دادیم، دنبال کنیم و پیش‌بینی‌های طبقه‌بندی را انجام دهیم.

```
# اجرای مدل و استنتاج
# Run inference
y_pred = []
for output in tqdm(pipe(KeyDataset(data["test"], "t5")),
total=len(data["test"])):
    text = output[0]["generated_text"]
    y_pred.append(0 if text == "negative" else 1)
```

در کد فوق، از آنجا که این مدل خروجی متنی تولید می‌کند، لازم بود که این خروجی‌ها به مقادیر عددی تبدیل شوند. به‌طور مشخص، کلمه‌ی «negative» به ۰ و کلمه‌ی «positive» به ۱ نگاشت داده شد.

این مقادیر عددی اکنون به ما امکان می‌دهند که کیفیت عملکرد مدل را همانند روش‌های قبلی مورد ارزیابی قرار دهیم.

ارزیابی عملکرد مدل

```
evaluate_performance(data["test"]["label"], y_pred)
```

نتایج به دست آمده به صورت زیر است:

	precision	recall	f1-score	support
Negative Review	0.83	0.85	0.84	533
Positive Review	0.85	0.83	0.84	533
accuracy			0.84	1066
macro avg	0.84	0.84	0.84	1066
weighted avg	0.84	0.84	0.84	1066

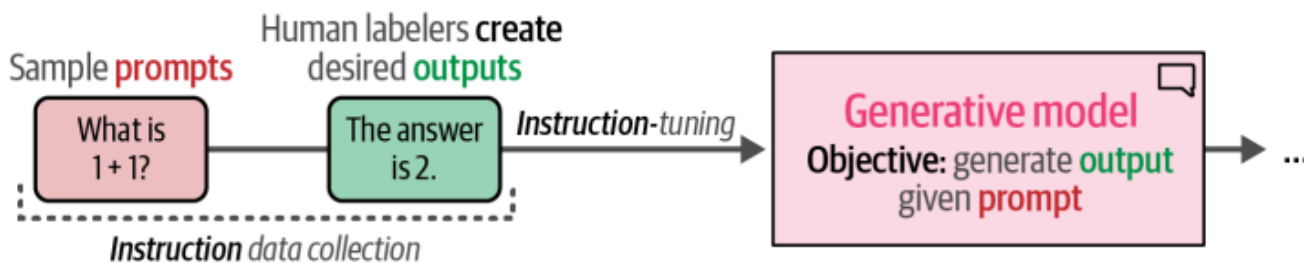
با کسب نمره **F1 برابر با ۰.۸۴**، واضح است که مدل **Flan-T5** نمونه‌ای بارز از توانمندی‌های **مدل‌های تولیدی (Generative Models)** است و نشان می‌دهد که این مدل‌ها حتی در ارزیابی‌های اولیه توانایی ارائه پیش‌بینی‌های دقیق و قابل اعتمادی را دارا هستند.

استفاده از ChatGPT برای طبقه‌بندی

اگرچه در این کتاب تمرکز اصلی بر **مدل‌های متن‌باز (Open Source)** است، ولی لازم به ذکر است که یکی دیگر از اجزای اصلی در حوزه هوش مصنوعی زبان، مدل‌های **اختصاصی و بسته (Closed Source)** است که **ChatGPT** نمونه‌ای برجسته از آن محسوب می‌شود.

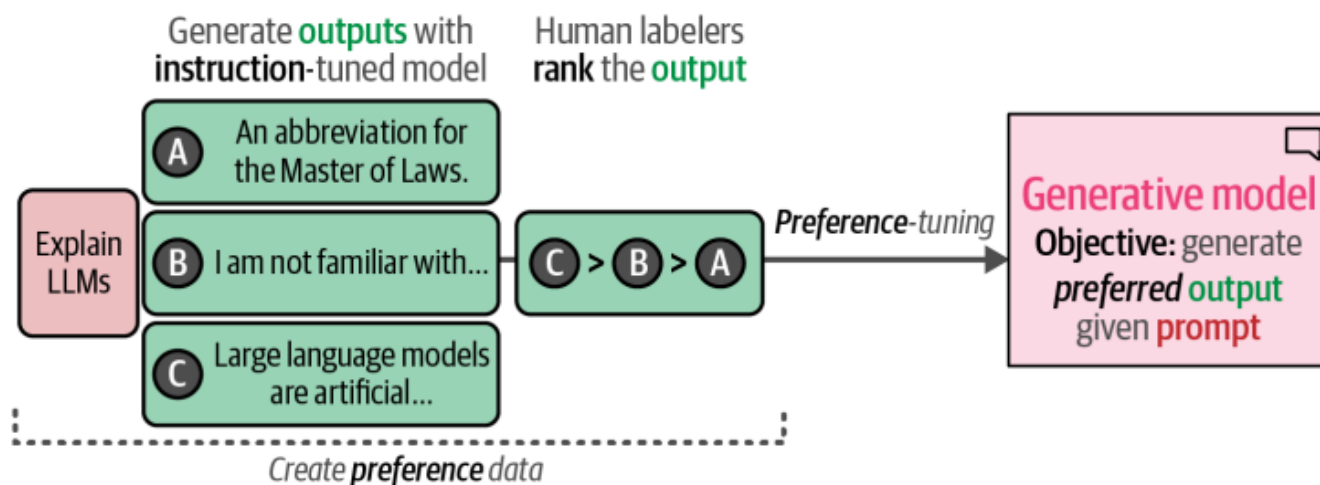
معماری پایه‌ی مدل اصلی ChatGPT (نسخه GPT-3.5) به‌طور رسمی منتشر نشده است، اما با توجه به نام آن می‌توان نتیجه گرفت که این مدل بر اساس معماری **فقط رمزگشا (Decoder-Only)** طراحی شده است؛ همان معماری‌ای که در مدل‌های پیشین GPT بررسی کرده‌ایم و به مدل اجازه می‌دهد متن ورودی را دریافت کرده و خروجی متنی تولید کند.

شرکت OpenAI در [نمای کلی](#) خود فرآیند آموزش مدل را تشریح کرده است که شامل یک مرحله‌ی کلیدی به نام **تنظیم بر اساس ترجیحات انسانی (Preference Tuning)** می‌شود. در این روش، ابتدا تیم OpenAI به صورت دستی خروجی‌های مطلوب (ایده‌آل) برای هر دستور ورودی (input prompt) را تهیه می‌کند و سپس از این داده‌ها برای ایجاد نسخه‌ی اولیه‌ی مدل استفاده می‌نماید. پس از آن، مدل بر اساس این داده‌ها آموزش می‌بیند تا پاسخی تولید کند که با **ترجیحات انسانی** هم‌خوانی داشته باشند. این فرآیند نه تنها باعث می‌شود مدل بتواند متن تولید کند، بلکه تضمین می‌کند که پاسخ‌ها مفید، مرتبط و مطابق با انتظارات کاربران باشند و دقت و کیفیت تعاملات انسانی با مدل به طور قابل توجهی افزایش یابد.



پس از ساخت نسخه‌ی اولیه‌ی مدل، شرکت OpenAI آن را برای تولید چندین پاسخ مختلف به یک ورودی (Prompt) مورد استفاده قرار داد. این پاسخ‌ها سپس توسط انسان‌ها به صورت دستی از بهترین تا ضعیف‌ترین

رتبه‌بندی شدند تا مشخص شود کدام پاسخ‌ها دقیق‌تر، مرتبط‌تر و مفهومی‌تر هستند. این رتبه‌بندی‌ها به‌عنوان داده‌های ترجیحی (**Preference Data**) جمع‌آوری و در آموزش نهایی مدل به‌کار گرفته شدند. به کمک این داده‌ها، مدل یاد گرفت پاسخ‌هایی تولید کند که با ترجیحات و انتظارات انسانی همخوانی داشته باشند. نتیجه‌ی این فرآیند، نسخه‌ی نهایی ChatGPT است که قادر است پاسخ‌هایی کاربردی، دقیق، مفید، مرتبط و هم‌راستا با انتظارات کاربران ارائه دهد.



یکی از مزایای برجسته استفاده از داده‌های ترجیحی (**Preference Data**) نسبت به داده‌های دستورالعملی (**Instruction Data**) این است که این داده‌ها می‌توانند تفاوت‌های ظریف بین پاسخ‌ها را به مدل منتقل کنند. به بیان دیگر، با ارائه نمونه‌هایی از یک پاسخ خوب و یک پاسخ بهتر، مدل تولیدی (**Generative Model**) یاد می‌گیرد که نه تنها پاسخ تولید کند، بلکه کیفیت پاسخ‌ها را به‌گونه‌ای بهبود دهد که با ترجیحات انسانی همخوانی داشته باشد. این فرآیند باعث می‌شود خروجی‌های مدل، دقیق‌تر، مرتبط‌تر و طبیعی‌تر به نظر برسند و تجربه کاربر بهبود یابد. در فصل‌های بعدی، به تفصیل روش‌های آموزش نهایی مدل (**Fine-Tuning**) و تنظیم مدل بر اساس ترجیحات انسانی (**Preference-Tuning**) بررسی می‌شوند و همچنین توضیح داده می‌شود که چگونه می‌توانید این مراحل را خودتان برای مدل‌های مختلف اجرا کنید. فرآیند استفاده از یک مدل اختصاصی (**Closed-Source**) تفاوت قابل‌توجهی با نمونه‌های متن‌باز (**Open-Source**) دارد که تاکنون بررسی کردیم. در این روش، به جای اینکه مدل را مستقیماً روی سیستم خود بارگذاری کنیم، دسترسی به مدل از طریق رابط برنامه‌نویسی اپلیکیشن (**API**) شرکت **OpenAI** صورت می‌گیرد. این رویکرد به ما امکان می‌دهد از مدل‌های پیشرفته بدون نیاز به منابع محاسباتی گسترده بهره‌مند شویم و تعامل با مدل را به‌سادگی مدیریت کنیم.

قبل از ورود به مثال طبقه‌بندی، ابتدا لازم است یک حساب کاربری رایگان در سایت [openai](https://openai.com) ایجاد کنید و سپس یک کلید **API** در [این لینک](#) بسازید. پس از انجام این مراحل، می‌توانید از طریق این کلید **API** با سرورهای **OpenAI** ارتباط برقرار کرده و مدل‌ها را فراخوانی کنید.

ما می‌توانیم از این کلید برای ایجاد یک کلاینت استفاده کنیم:

```
import openai
# Create client
client = openai.OpenAI(api_key="YOUR_KEY_HERE")
```


با استفاده از این کلاینت، ما تابعی به نام **chatgpt_generation** ایجاد می‌کنیم که به ما اجازه می‌دهد متنی را بر اساس یک دستور (Prompt) مشخص، سند ورودی و مدل انتخاب‌شده تولید کنیم:

```
def chatgpt_generation(prompt, document, model="gpt-3.5-turbo-0125"):
    """Generate an output based on a prompt and an input document."""
    messages=[
        { "role": "system",
          "content": "You are a helpful assistant."
        },
        { "role": "user",
          "content":
            prompt.replace("[DOCUMENT]", document)
        }
    ]
    chat_completion = client.chat.completions.create(
        messages=messages,
        model=model,
        temperature=0
    )
    return chat_completion.choices[0].message.content
```

در گام بعد، لازم است یک **قالب (Template)** ایجاد کنیم تا از مدل بخواهیم عملیات **طبقه‌بندی** را برای ما انجام دهد. این قالب مشخص می‌کند که مدل چگونه باید ورودی را پردازش کرده و خروجی متناسب با کلاس‌های مورد نظر تولید کند.

```
# Define a prompt template as a base
prompt = """Predict whether the following document is a positive or negative
movie review:
[DOCUMENT]
If it is positive return 1 and if it is negative return 0. Do not give any
other answers.
"""

# Predict the target using GPT
document = "unpretentious , charming , quirky , original"
chatgpt_generation(prompt, document)
```

این **قالب (Template)** تنها یک نمونه‌ی اولیه است و می‌توان آن را به دلخواه تغییر داد. در اینجا، آن را به ساده‌ترین شکل ممکن طراحی کرده‌ایم تا نحوه استفاده از قالب برای راهنمایی مدل به‌روشنی و به‌صورت قابل فهم نشان داده شود.

پیش از استفاده از این روش روی یک **مجموعه داده بزرگ**، بسیار مهم است که همیشه میزان مصرف خود را زیر نظر داشته باشید. استفاده از **API‌های خارجی** مانند سرویس OpenAI می‌تواند در صورت انجام درخواست‌های متعدد به سرعت هزینه‌بر شود. در زمان نگارش این متن، اجرای **مجموعه داده آزمایشی** با مدل

”gpt-3.5-turbo-0125” حدود ۳ سنت هزینه دارد که توسط حساب رایگان پوشش داده می‌شود، اما این مقدار ممکن است در آینده تغییر کند.

هنگام کار با API‌های خارجی ممکن است با خطاهای محدودیت نرخ (Rate Limit Errors) مواجه شوید. این خطاها زمانی رخ می‌دهند که بیش از حد مجاز در یک بازه زمانی مشخص، درخواست به API ارسال کنید؛ چرا که برخی API‌ها تعداد درخواست‌های قابل انجام در دقیقه یا ساعت را محدود می‌کنند. برای جلوگیری از این خطاها، می‌توان از روش‌های مختلفی برای تلاش مجدد (Retrying) درخواست استفاده کرد، از جمله تکنیکی به نام Exponential Backoff. در این روش، هر بار که با خطای محدودیت نرخ مواجه می‌شویم، مدتی کوتاه صبر (Sleep) انجام می‌شود و سپس درخواست ناموفق دوباره ارسال می‌گردد. اگر درخواست دوباره ناموفق بود، طول مدت صبر افزایش می‌یابد تا زمانی که درخواست موفق شود یا به حداکثر تعداد تلاش‌ها برسیم. برای استفاده از این روش با OpenAI، راهنمای جامعی وجود دارد که می‌تواند شما را در شروع کار راهنمایی کند.

در گام بعد، می‌توانیم این فرآیند را روی تمام نقدهای موجود در مجموعه داده آزمایشی (Test Dataset) اجرا کنیم تا پیش‌بینی‌های مدل را دریافت کنیم. در صورتی که مایل باشید اعتبار رایگان خود را برای سایر وظایف ذخیره کنید، می‌توانید از اجرای این مرحله صرف‌نظر کنید.

```
# You can skip this if you want to save your (free) credits
predictions = [
    chatgpt_generation(prompt, doc) for doc in tqdm(data["test"]["text"])
]
```

همانند مثال قبلی، برای ارزیابی عملکرد مدل لازم است خروجی‌ها که به صورت رشته‌ای (String) هستند را به اعداد صحیح (Integer) تبدیل کنیم.

```
# Extract predictions
y_pred = [int(pred) for pred in predictions]
# Evaluate performance
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.87	0.97	0.92	533
Positive Review	0.96	0.86	0.91	533
accuracy			0.91	1066
macro avg	0.92	0.91	0.91	1066
weighted avg	0.92	0.91	0.91	1066

امتیاز F1 برابر با ۰.۹۱ نشان‌دهنده توانمندی قابل توجه مدل است، مدلی که هوش مصنوعی تولیدی را در دسترس عموم قرار داد. با این حال، از آنجا که اطلاعاتی از داده‌های مورد استفاده در آموزش مدل نداریم، نمی‌توان به‌سادگی از این معیارها برای ارزیابی دقیق آن استفاده کرد. ممکن است مدل حتی روی همین مجموعه داده ما آموزش دیده باشد!

در فصل ۱۲، به بررسی روش‌هایی خواهیم پرداخت که بتوانیم مدل‌های متن‌باز و مدل‌های اختصاصی (Closed Source) را روی وظایف عمومی‌تر و گسترده‌تر ارزیابی کنیم.

خلاصه

در این فصل، به بررسی روش‌های متنوع طبقه‌بندی متن پرداختیم؛ از آموزش کامل مدل‌ها (Fine-Tuning) گرفته تا استفاده از مدل‌های از پیش آموزش‌دیده بدون نیاز به تنظیمات اضافی. دریافتیم که طبقه‌بندی متن فرایندی ساده نیست و برای انجام دقیق آن، نیاز به تکنیک‌های متنوع و خلاقانه‌ای داریم.

دو دسته اصلی مدل‌های زبانی که بررسی شدند عبارت‌اند از: مدل‌های نمایشی (Representation Models) و مدل‌های تولیدی (Generative Models).

در مدل‌های نمایشی، دو نوع مدل استفاده شد:

- **مدل خاص وظیفه (Task-Specific Model):** این مدل به‌طور ویژه برای تحلیل احساسات آموزش دیده بود و نشان داد که مدل‌های از پیش آموزش‌دیده می‌توانند در طبقه‌بندی متون عملکرد بسیار مؤثری داشته باشند.
- **مدل امبدینگ (Embedding Model):** این مدل ویژگی‌های عددی چندمنظوره تولید می‌کند و می‌تواند به عنوان ورودی برای یک طبقه‌بند سبک و سریع مانند رگرسیون لجستیک به کار رود. با این روش، بدون نیاز به آموزش کامل مدل، می‌توان یک طبقه‌بند کارآمد و سریع ساخت.

در مدل‌های تولیدی نیز دو نوع بررسی شد:

- **مدل متن‌باز Encoder-Decoder** مانند **Flan-T5**،
- **مدل اختصاصی Decoder-Only** مانند **GPT-3.5**.

این مدل‌ها نشان دادند که حتی بدون داده‌های برچسب‌گذاری‌شده، می‌توان متون را با دقت قابل توجهی طبقه‌بندی کرد و توانایی بالای مدل‌های تولیدی در انجام **وظایف متنوع طبقه‌بندی** را به خوبی به نمایش گذاشتند.

در فصل بعد، تمرکز ما بر **طبقه‌بندی بدون ناظر (Unsupervised Classification)** خواهد بود. در این حالت، اگر داده‌های متنی بدون برچسب در اختیار داشته باشیم، هدف شناسایی الگوها و ساختارهای پنهان در داده‌ها است. برای این منظور از روش‌های **خوشه‌بندی (Clustering)** و **مدل‌سازی موضوعی (Topic Modeling)** استفاده خواهیم کرد تا بتوانیم خوشه‌ها را شناسایی و نام‌گذاری کنیم.

"Pasted image 20260226112417.png1" could not be found.

"Pasted image 20260226112253.png1" could not be found.

"Pasted image 20260226112304.png1" could not be found.

"Pasted image 20260226112322.png1" could not be found.

"Pasted image 20260226112330.png1" could not be found.

"Pasted image 20260226112337.png1" could not be found.

"Pasted image 20260226112345.png1" could not be found.

"Pasted image 20260226112351.png1" could not be found.

"Pasted image 20260226112424.png1" could not be found.

"Pasted image 20260226112433.png1" could not be found.

"Pasted image 20260226112439.png1" could not be found.

"Pasted image 20260226112446.png1" could not be found.

"Pasted image 20260226112522.png1" could not be found.

"Pasted image 20260226112527.png1" could not be found.

"Pasted image 20260226112534.png1" could not be found.